



Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

AI for IT Security Professionals

TGS Ref No: TGS-2023039344

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version v6.0

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
v5	Legacy reference	Previous Azure course deck and supporting material	Tertiary Infotech
v6.0	6 September 2026	Full AI security rebuild; 16 self-contained labs; preserved live assessment structure	Tertiary Infotech

TABLE OF CONTENTS

Course purpose	4
Learning outcomes	4
Schedule and assessment	4
Access course material	4
Environment and evidence rules	4
Lab 01 Risk register and patch rollout	5
Lab 02 Redaction and AI evidence validation	8
Lab 03 RBAC and privileged access	11
Lab 04 Conditional Access and workload identity	15
Lab 05 NSG segmentation and flow diagnosis	19
Lab 06 Web edge and private access	23
Lab 07 VM administration and encryption	27
Lab 08 Storage SQL and secret protection	31
Lab 09 Policy and security posture	35
Lab 10 Security logs and detection metrics	39
Lab 11 Access rights troubleshooting	43
Lab 12 Unauthorized access investigation	46
Lab 13 Prompt injection and retrieval boundaries	49
Lab 14 Tool gateway and response approvals	52
Lab 15 AI security evaluation	55
Lab 16 Incident response and service recovery	58
Source register	60

Course purpose

This guide develops evidence-based security administration across identity, networks, data, monitoring and AI-assisted operations. You will inspect technical mechanisms, run controlled synthetic experiments, verify AI claims and record operational evidence. The PowerPoint explains mechanisms and evidence; the detailed procedures are here and in each lab folder.

Learning outcomes

LO1: Administer security programmes and analyze the impact of system updates.

LO2: Perform system administration and configure network device security features.

LO3: Perform troubleshooting of security software and assist users in defining access rights.

LO4: Coordinate access control rights and investigate unauthorized access incidents.

Schedule and assessment

Four days, 32 contact hours in total: 15 hours facilitated learning, 15 hours practical work and 2 hours assessment. Days 1–3 each contain 8 training hours; Day 4 contains 6 training hours and WA 60 minutes plus PP 60 minutes. Breaks are additional and are excluded from contact hours. The written paper contains nine open-ended questions K1–K9. The practical paper contains three tasks A1–A6. The assessor evaluates demonstrated competence; a permission blocker is not itself evidence that a control works.

Access course material

1. Open <https://lms-tms.tertiaryinfotech.com/> and sign in with your registered account.
2. Select AI for IT Security Professionals and confirm TGS-2023039344.
3. Open the courseware section and download the learner slides, Learner Guide and lab resources.
4. Keep each lab folder intact. Open its INSTRUCTIONS.md or INSTRUCTIONS.pdf before running analyse.py.
5. Submit completed assessment documents through the assigned LMS assessment area.

Environment and evidence rules

Use Python 3.10 or later. The standard-library scripts run offline on Windows, macOS or Linux. They neither train nor invoke AI models and never connect to Azure. The separate AI review prompt is for an organisation-approved tool and must use synthetic data only. Azure verification uses a trainer-provided sandbox. Label every result as simulation, operationally verified or blocked. Preserve original input files, hashes, record IDs and UTC timestamps. Never upload tokens, secrets, personal identifiers or production logs to an unapproved tool.

Lab 01 Risk register and patch rollout

AI for IT Security Professionals | TGS-2023039344 | v6.0

Purpose and output

Create a ranked risk register and canary decision. This lab supports LO1. It uses a simplified deterministic teaching model, not a trained AI system or full Azure emulator. The AI review is a separate exercise using the included prompt.

Requirements

Python 3.10 or later, a text editor and this entire folder. No packages, network access or API key are required for the baseline. On Windows use `py -3` if `python3` is unavailable. An approved AI tool is optional; the trainer can provide a review response for critique. Azure exercises require the trainer-assigned subscription, permission and feature entitlement. Record an exact blocker where unavailable; never describe an unperformed test as passed.

Folder contents

- `mock-data.json`: synthetic input with no real personal data or credentials.
- `analyse.py`: runnable analysis for this lab only.
- `expected-results.json`: expected baseline and source checksum.
- `ai-review-prompt.txt`: evidence-constrained AI review prompt.
- `evidence-template.md`: your deliverable template.
- `INSTRUCTIONS.md` and `INSTRUCTIONS.pdf`: the same procedure in two formats.

1 Prepare a working copy

Copy this whole folder to a location you can edit. Open a terminal in the copied folder. Confirm Python and inspect the data before running code.

```
python3 --version
python3 -m json.tool mock-data.json
```

Identify the record IDs and explain the meaning of each field. All values are invented classroom fixtures. Open `analyse.py` in the editor and locate the decision rule. It reads a local JSON file and writes a local report.

2 Run the baseline

```
python3 analyse.py --input mock-data.json --output results.json
```

Open `results.json`. The expected decisions in output order are: ROLLBACK, CONTINUE, CONTINUE. The `source_sha256` binds the report to the exact input bytes. Compare against the supplied baseline:

```
python3 -c "import json; a=json.load(open('results.json')); b=json.load(open('expected-results.json')); assert a==b; print('BASELINE PASS')"
```

A pass proves this fixture was processed as specified; it does not prove an Azure control was deployed. Copy the input checksum and decision rows into your evidence template.

3 Trace the mechanism

Asset ownership

Join security findings to an owned asset before prioritising remediation.

Contract: `asset_id`, `owner`, `internet_exposed`, `criticality`

Accepted case: srv-pay has owner Priya and criticality 5.

Counterexample: srv-lab has no accountable owner.

Diagnosis: Quarantine the unmapped finding from automated assignment; resolve the inventory gap.

Evidence: Record inventory version, join key and unmatched row count.

Patch prioritisation

Rank exposed high-impact assets before isolated low-impact assets.

Contract: $\text{score} = \text{severity} * \text{criticality} * \text{exposure_weight}$

Accepted case: $8 * 5 * 2 = 80$ for internet-facing srv-pay.

Counterexample: $9 * 1 * 1 = 9$ for isolated srv-lab.

Diagnosis: A severity-only queue misses business impact; validate compensating controls before ordering changes.

Evidence: Retain component values and the final priority order; this is a classroom heuristic, not CVSS.

Change and rollback gates

A canary release moves to wider rollout only after error and health checks.

Contract: change_id, baseline_error_rate, canary_error_rate, rollback_owner

Accepted case: 0.5% baseline and 0.6% canary meet a 1 percentage-point guard.

Counterexample: 0.5% baseline and 3% canary breach the guard.

Diagnosis: Pause rollout, restore the previous approved image and re-test; do not merely close the ticket.

Evidence: Capture pre/post health evidence and approval state for CHG-104.

4 Change one input and test the consequence

Save a copy of mock-data.json as changed-data.json using the editor. Change srv-pay canary from 3.0 to 0.6. Its decision must change from ROLLBACK to CONTINUE while its risk score remains 80.

```
python3 analyse.py --input changed-data.json --output changed-results.json
```

Compare decisions and source hashes with the baseline. Identify the exact expression in analyse.py responsible for the changed behaviour. Do not overwrite expected-results.json to make a failing baseline pass. Restore the original fixture if you edited it accidentally.

5 Review AI claims against evidence

Open ai-review-prompt.txt. In an organisation-approved AI tool, submit the prompt with the synthetic input and result only. Record the tool/model name, date and prompt version if available. If no approved AI tool is available, manually draft a tentative analyst summary and label it as a human exercise.

For each returned claim, record its cited ID, the actual field value, whether the claim follows, and any correction. Reject conclusions unsupported by the records even when the model is confident. Include one alternative explanation. The AI response is a proposal; it has no authority to approve or execute a security action.

6 Verify the corresponding operational control

Azure Portal > Resource groups > select the training group > Activity log. Open a change event and record its operation, timestamp and correlation ID. Review the affected resource configuration before and after the approved change. Do not deploy a patch outside the assigned sandbox.

For every screenshot or export, record resource scope, UTC time and expected versus observed result. Redact personal data and secrets. If blocked, record the exact error, missing permission/licence, what could be inspected, and the unverified outcome. Ask the trainer to provide an authorised sandbox demonstration where the assessed ability cannot otherwise be evidenced.

7 Submit evidence and clean up

Complete evidence-template.md with baseline, changed result, AI claim review and Azure evidence or clearly labelled limitations. Keep results.json and changed-results.json with your submission. Check that your conclusion distinguishes an observed fact from a hypothesis. Remove only resources or assignments you created with trainer approval; do not delete shared training infrastructure.

Acceptance checks

- The unchanged fixture produces the exact expected report.
- The changed fixture produces the explained result and a different input hash.
- At least one AI or analyst claim is checked against a specific record and field.
- The report states simulation, Azure verified or blocked for each relevant claim.
- The output is a ranked risk register and canary decision with an owner, evidence and remaining uncertainty.

Troubleshooting

- Python not found: install the trainer-approved Python distribution or use py -3 on Windows.
- File not found: open the terminal in this lab folder; check the input filename.
- JSON parse error: validate with `python3 -m json.tool changed-data.json`; use lowercase true and false in JSON.
- Baseline mismatch: restore the supplied fixture and inspect the first differing decision; never edit the expected file.
- Azure denial or missing feature: capture the exact error and scope. A blocker is a limitation, not proof of competence or deployment.
- AI invents evidence: reject the claim, cite the missing ID, and request an evidence-limited revision.

References

The Learner Guide contains the complete source register and the lecture explanations for this lab. Original examples are adapted from the supplied Azure and AI-security references. Product behaviour should be checked against current Microsoft Learn documentation before a real deployment.

Lab 02 Redaction and AI evidence validation

AI for IT Security Professionals | TGS-2023039344 | v6.0

Purpose and output

Create a redacted evidence table and corrected AI claims. This lab supports LO1. It uses a simplified deterministic teaching model, not a trained AI system or full Azure emulator. The AI review is a separate exercise using the included prompt.

Requirements

Python 3.10 or later, a text editor and this entire folder. No packages, network access or API key are required for the baseline. On Windows use `py -3` if `python3` is unavailable. An approved AI tool is optional; the trainer can provide a review response for critique. Azure exercises require the trainer-assigned subscription, permission and feature entitlement. Record an exact blocker where unavailable; never describe an unperformed test as passed.

Folder contents

- `mock-data.json`: synthetic input with no real personal data or credentials.
- `analyse.py`: runnable analysis for this lab only.
- `expected-results.json`: expected baseline and source checksum.
- `ai-review-prompt.txt`: evidence-constrained AI review prompt.
- `evidence-template.md`: your deliverable template.
- `INSTRUCTIONS.md` and `INSTRUCTIONS.pdf`: the same procedure in two formats.

1 Prepare a working copy

Copy this whole folder to a location you can edit. Open a terminal in the copied folder. Confirm Python and inspect the data before running code.

```
python3 --version
python3 -m json.tool mock-data.json
```

Identify the record IDs and explain the meaning of each field. All values are invented classroom fixtures. Open `analyse.py` in the editor and locate the decision rule. It reads a local JSON file and writes a local report.

2 Run the baseline

```
python3 analyse.py --input mock-data.json --output results.json
```

Open `results.json`. The expected decisions in output order are: `SUPPORTED_ID`, `UNSUPPORTED_ID`, `SUPPORTED_ID`. The `source_sha256` binds the report to the exact input bytes. Compare against the supplied baseline:

```
python3 -c "import json; a=json.load(open('results.json')); b=json.load(open('expected-results.json')); assert a==b; print('BASELINE PASS')"
```

A pass proves this fixture was processed as specified; it does not prove an Azure control was deployed. Copy the input checksum and decision rows into your evidence template.

3 Trace the mechanism

Evidence classification

Classify a finding as observed, inferred or unsupported before accepting an AI summary.

Contract: `claim_id`, `evidence_ids`, `conclusion`, `confidence`

Accepted case: Claim C1 cites event E17 with a matching field.

Counterexample: Claim C2 names event E99 absent from the dataset.

Diagnosis: Return unsupported claims for correction; confidence text does not establish provenance.

Evidence: Record the missing evidence identifier and corrected wording.

Telemetry redaction

Replace personal identifiers before logs leave the approved security boundary.

Contract: user_id, src_ip, token, tenant_id

Accepted case: user_004 replaces a real address and token is removed.

Counterexample: A bearer token remains in the prompt body.

Diagnosis: Block submission, remove the secret, and rotate an exposed live credential through the incident process.

Evidence: Keep the redaction rule and a synthetic before/after example.

Control responsibility

Assign each cloud control to a named operator and review its implementation evidence.

Contract: control_id, accountable, evidence_uri, review_due

Accepted case: CTRL-7 has an owner and a dated access review.

Counterexample: A policy statement has no operational evidence.

Diagnosis: Separate policy intent from deployed state; request a configuration export and a negative test.

Evidence: A control register must show evidence date and unresolved exceptions.

4 Change one input and test the consequence

Save a copy of mock-data.json as changed-data.json using the editor. Change C2 evidence_id to E18. Identifier validation will pass, but the claim still does not prove compromise. Explain why matching an ID is necessary but insufficient.

```
python3 analyse.py --input changed-data.json --output changed-results.json
```

Compare decisions and source hashes with the baseline. Identify the exact expression in analyse.py responsible for the changed behaviour. Do not overwrite expected-results.json to make a failing baseline pass. Restore the original fixture if you edited it accidentally.

5 Review AI claims against evidence

Open ai-review-prompt.txt. In an organisation-approved AI tool, submit the prompt with the synthetic input and result only. Record the tool/model name, date and prompt version if available. If no approved AI tool is available, manually draft a tentative analyst summary and label it as a human exercise.

For each returned claim, record its cited ID, the actual field value, whether the claim follows, and any correction. Reject conclusions unsupported by the records even when the model is confident. Include one alternative explanation. The AI response is a proposal; it has no authority to approve or execute a security action.

6 Verify the corresponding operational control

In the approved log viewer, inspect the record schema without exporting personal data. Record which fields are confidential. Use only the supplied synthetic rows for the AI exercise; identifier existence does not validate a claim's meaning.

For every screenshot or export, record resource scope, UTC time and expected versus observed result. Redact personal data and secrets. If blocked, record the exact error, missing permission/licence, what could be inspected, and the unverified outcome. Ask the trainer to provide an authorised sandbox demonstration where the assessed ability cannot otherwise be evidenced.

7 Submit evidence and clean up

Complete evidence-template.md with baseline, changed result, AI claim review and Azure evidence or clearly labelled limitations. Keep results.json and changed-results.json with your submission. Check that your conclusion distinguishes an observed fact from a hypothesis. Remove only resources or assignments you created with trainer approval; do not delete shared training infrastructure.

Acceptance checks

- The unchanged fixture produces the exact expected report.
- The changed fixture produces the explained result and a different input hash.
- At least one AI or analyst claim is checked against a specific record and field.
- The report states simulation, Azure verified or blocked for each relevant claim.
- The output is a redacted evidence table and corrected AI claims with an owner, evidence and remaining uncertainty.

Troubleshooting

- Python not found: install the trainer-approved Python distribution or use py -3 on Windows.
- File not found: open the terminal in this lab folder; check the input filename.
- JSON parse error: validate with `python3 -m json.tool changed-data.json`; use lowercase true and false in JSON.
- Baseline mismatch: restore the supplied fixture and inspect the first differing decision; never edit the expected file.
- Azure denial or missing feature: capture the exact error and scope. A blocker is a limitation, not proof of competence or deployment.
- AI invents evidence: reject the claim, cite the missing ID, and request an evidence-limited revision.

References

The Learner Guide contains the complete source register and the lecture explanations for this lab. Original examples are adapted from the supplied Azure and AI-security references. Product behaviour should be checked against current Microsoft Learn documentation before a real deployment.

Lab 03 RBAC and privileged access

AI for IT Security Professionals | TGS-2023039344 | v6.0

Purpose and output

Create a access matrix with a least-privilege change proposal. This lab supports LO3. It uses a simplified deterministic teaching model, not a trained AI system or full Azure emulator. The AI review is a separate exercise using the included prompt.

Requirements

Python 3.10 or later, a text editor and this entire folder. No packages, network access or API key are required for the baseline. On Windows use `py -3` if `python3` is unavailable. An approved AI tool is optional; the trainer can provide a review response for critique. Azure exercises require the trainer-assigned subscription, permission and feature entitlement. Record an exact blocker where unavailable; never describe an unperformed test as passed.

Folder contents

- `mock-data.json`: synthetic input with no real personal data or credentials.
- `analyse.py`: runnable analysis for this lab only.
- `expected-results.json`: expected baseline and source checksum.
- `ai-review-prompt.txt`: evidence-constrained AI review prompt.
- `evidence-template.md`: your deliverable template.
- `INSTRUCTIONS.md` and `INSTRUCTIONS.pdf`: the same procedure in two formats.

1 Prepare a working copy

Copy this whole folder to a location you can edit. Open a terminal in the copied folder. Confirm Python and inspect the data before running code.

```
python3 --version
python3 -m json.tool mock-data.json
```

Identify the record IDs and explain the meaning of each field. All values are invented classroom fixtures. Open `analyse.py` in the editor and locate the decision rule. It reads a local JSON file and writes a local report.

2 Run the baseline

```
python3 analyse.py --input mock-data.json --output results.json
```

Open `results.json`. The expected decisions in output order are: `SCOPED`, `REVIEW_BROAD`, `SCOPED`. The `source_sha256` binds the report to the exact input bytes. Compare against the supplied baseline:

```
python3 -c "import json; a=json.load(open('results.json')); b=json.load(open('expected-results.json')); assert a==b; print('BASELINE PASS')"
```

A pass proves this fixture was processed as specified; it does not prove an Azure control was deployed. Copy the input checksum and decision rows into your evidence template.

3 Trace the mechanism

RBAC scope inheritance

Permissions at parent scope apply to child resources; additional Reader access does not reduce Contributor.

Contract: principalId, roleDefinitionId, scope

Accepted case: Reader at rg-sec provides read-only management access.

Counterexample: Contributor at subscription remains effective below rg-sec.

Diagnosis: Inspect all applicable assignments and deny assignments before changing local roles.

Evidence: Preserve principal ID, full scope and inherited role list.

Custom role actions

Management actions and data actions authorise different operations; subtracting NotActions is not a global deny.

Contract: Actions, NotActions, DataActions, AssignableScopes

Accepted case: A custom inventory role grants

Microsoft.Resources/subscriptions/resourceGroups/read.

Counterexample: Granting Microsoft.Storage/storageAccounts/listKeys/action exposes account keys.

Diagnosis: Remove unnecessary key-list access and review other assignments that may still grant it.

Evidence: Compare allowed action strings against the approved job duties.

Privileged activation

An eligible privileged role becomes active for an approved bounded interval.

Contract: principal, role, justification, start_utc, end_utc

Accepted case: An approved 30-minute activation expires at 10:30Z.

Counterexample: A standing Owner assignment has no expiry.

Diagnosis: Revoke unnecessary standing access; verify emergency access separately from ordinary roles.

Evidence: Record eligible versus active state, approver and actual expiry.

4 Change one input and test the consequence

Save a copy of mock-data.json as changed-data.json using the editor. Add Reader to u-owner. The result must remain REVIEW_BROAD because a narrower grant does not remove Owner.

```
python3 analyse.py --input changed-data.json --output changed-results.json
```

Compare decisions and source hashes with the baseline. Identify the exact expression in analyse.py responsible for the changed behaviour. Do not overwrite expected-results.json to make a failing baseline pass. Restore the original fixture if you edited it accidentally.

5 Review AI claims against evidence

Open ai-review-prompt.txt. In an organisation-approved AI tool, submit the prompt with the synthetic input and result only. Record the tool/model name, date and prompt version if available. If no approved AI tool is available, manually draft a tentative analyst summary and label it as a human exercise.

For each returned claim, record its cited ID, the actual field value, whether the claim follows, and any correction. Reject conclusions unsupported by the records even when the model is confident. Include one alternative explanation. The AI response is a proposal; it has no authority to approve or execute a security action.

6 Verify the corresponding operational control

Azure procedure A Establish the sandbox and role evidence

1. Sign in to Azure Portal. Use the directory/subscription selector to choose the trainer-approved subscription. Record its name and ID; stop if the selected subscription is not the training allocation.
2. Open Resource groups > Create. Select the assigned subscription and region. Name the group rg-itsec-<initials> and create it. Record the resulting full resource-group ID.
3. Open the new group > Access control (IAM) > Check access. Select the approved test principal. Record every direct and inherited role, including the scope of parent assignments.
4. With Role Based Access Control Administrator or another authorised role-assignment permission, choose Add role assignment, select Reader, choose the test principal and use the resource-group scope. Review and assign. If unavailable, record the exact denied operation and request an authorised trainer demonstration.
5. Confirm the assignment appears. Using the approved test principal, verify resource inspection succeeds and an ungranted write operation is denied. A principal with inherited Contributor cannot serve as a valid negative-test subject; explain why and select the correct sandbox principal with the trainer.
6. Open IAM > Add > Add custom role if authorised. Name it ITSEC Inventory Reader <initials>. In Permissions, add only Microsoft.Resources/subscriptions/resourceGroups/read. Set Assignable scopes to the training resource group. Review the JSON before creation. Do not grant wildcard write or storage-key access. Record the role definition, assignment scope and its limited permitted action.
7. Compare the custom role with the legacy storage-key reader requirement. Locate Microsoft.Storage/storageAccounts/listKeys/action in a role definition and explain that access to account keys can enable broad data access. Demonstrate inspection of the action; add it only if the trainer explicitly allocates an isolated storage-key exercise. Do not capture key values.
8. Microsoft Entra admin center > Identity governance > Privileged Identity Management > Azure resources: select the training scope. Review eligible and active assignments. Where allocated, request a short approved activation with a justification. Record activation and expiry; otherwise record the licence/permission blocker and trainer-demonstrated evidence.
9. Save role JSON or screenshots, the test principal identifier, full scopes, actual allow/deny results and UTC times. Remove only the Reader/custom-role assignments created for this lab after the trainer confirms evidence is complete. Keep the resource group for later labs.

For every screenshot or export, record resource scope, UTC time and expected versus observed result. Redact personal data and secrets. If blocked, record the exact error, missing permission/licence, what could be inspected, and the unverified outcome. Ask the trainer to provide an authorised sandbox demonstration where the assessed ability cannot otherwise be evidenced.

7 Submit evidence and clean up

Complete evidence-template.md with baseline, changed result, AI claim review and Azure evidence or clearly labelled limitations. Keep results.json and changed-results.json with your submission. Check that your conclusion distinguishes an observed fact from a hypothesis. Remove only resources or assignments you created with trainer approval; do not delete shared training infrastructure.

Acceptance checks

- The unchanged fixture produces the exact expected report.

- The changed fixture produces the explained result and a different input hash.
- At least one AI or analyst claim is checked against a specific record and field.
- The report states simulation, Azure verified or blocked for each relevant claim.
- The output is a access matrix with a least-privilege change proposal with an owner, evidence and remaining uncertainty.

Troubleshooting

- Python not found: install the trainer-approved Python distribution or use py -3 on Windows.
- File not found: open the terminal in this lab folder; check the input filename.
- JSON parse error: validate with `python3 -m json.tool changed-data.json`; use lowercase true and false in JSON.
- Baseline mismatch: restore the supplied fixture and inspect the first differing decision; never edit the expected file.
- Azure denial or missing feature: capture the exact error and scope. A blocker is a limitation, not proof of competence or deployment.
- AI invents evidence: reject the claim, cite the missing ID, and request an evidence-limited revision.

References

The Learner Guide contains the complete source register and the lecture explanations for this lab. Original examples are adapted from the supplied Azure and AI-security references. Product behaviour should be checked against current Microsoft Learn documentation before a real deployment.

Lab 04 Conditional Access and workload identity

AI for IT Security Professionals | TGS-2023039344 | v6.0

Purpose and output

Create a policy-mode comparison and workload identity diagnosis. This lab supports LO3. It uses a simplified deterministic teaching model, not a trained AI system or full Azure emulator. The AI review is a separate exercise using the included prompt.

Requirements

Python 3.10 or later, a text editor and this entire folder. No packages, network access or API key are required for the baseline. On Windows use `py -3` if `python3` is unavailable. An approved AI tool is optional; the trainer can provide a review response for critique. Azure exercises require the trainer-assigned subscription, permission and feature entitlement. Record an exact blocker where unavailable; never describe an unperformed test as passed.

Folder contents

- `mock-data.json`: synthetic input with no real personal data or credentials.
- `analyse.py`: runnable analysis for this lab only.
- `expected-results.json`: expected baseline and source checksum.
- `ai-review-prompt.txt`: evidence-constrained AI review prompt.
- `evidence-template.md`: your deliverable template.
- `INSTRUCTIONS.md` and `INSTRUCTIONS.pdf`: the same procedure in two formats.

1 Prepare a working copy

Copy this whole folder to a location you can edit. Open a terminal in the copied folder. Confirm Python and inspect the data before running code.

```
python3 --version
python3 -m json.tool mock-data.json
```

Identify the record IDs and explain the meaning of each field. All values are invented classroom fixtures. Open `analyse.py` in the editor and locate the decision rule. It reads a local JSON file and writes a local report.

2 Run the baseline

```
python3 analyse.py --input mock-data.json --output results.json
```

Open `results.json`. The expected decisions in output order are: `WOULD_BLOCK`, `BLOCK`, `WRONG_AUDIENCE`. The `source_sha256` binds the report to the exact input bytes. Compare against the supplied baseline:

```
python3 -c "import json; a=json.load(open('results.json')); b=json.load(open('expected-results.json')); assert a==b; print('BASELINE PASS')"
```

A pass proves this fixture was processed as specified; it does not prove an Azure control was deployed. Copy the input checksum and decision rows into your evidence template.

3 Trace the mechanism

Conditional Access evaluation

Report-only evaluates a policy without enforcing its access decision.

Contract: `state=enabledForReportingButNotEnforced`

Accepted case: An MFA policy records a would-block result for the test user.

Counterexample: A report-only result is presented as an enforced block.

Diagnosis: Pilot enforcement with a scoped group after reviewing emergency-access exclusions.

Evidence: Preserve policy mode and sign-in evaluation result, not only its name.

Managed identity token flow

An Azure workload requests a token and the resource checks its audience and assigned permissions.

Contract: principal_id, token_audience, resource_scope

Accepted case: A workload identity has the required role on its target resource.

Counterexample: A valid token targets the wrong resource audience.

Diagnosis: Diagnose token audience separately from resource permission; never paste access tokens into AI tools.

Evidence: Record identity object ID, target resource and redacted error code.

Application consent boundaries

API permission consent and Azure resource RBAC are separate grants.

Contract: app_id, service_principal_id, permission_type, consent

Accepted case: An app has only the delegated scope needed for the signed-in user.

Counterexample: Tenant-wide application permission is mistaken for a user-only grant.

Diagnosis: Review whether app-only access is required and remove unused consent through approval.

Evidence: Capture permission type, consenting authority and intended data boundary.

4 Change one input and test the consequence

Save a copy of mock-data.json as changed-data.json using the editor. Set signin-1 mfa to true. The result changes to ALLOW in this simplified model; explain why this does not prove a real Conditional Access deployment.

```
python3 analyse.py --input changed-data.json --output changed-results.json
```

Compare decisions and source hashes with the baseline. Identify the exact expression in analyse.py responsible for the changed behaviour. Do not overwrite expected-results.json to make a failing baseline pass. Restore the original fixture if you edited it accidentally.

5 Review AI claims against evidence

Open ai-review-prompt.txt. In an organisation-approved AI tool, submit the prompt with the synthetic input and result only. Record the tool/model name, date and prompt version if available. If no approved AI tool is available, manually draft a tentative analyst summary and label it as a human exercise.

For each returned claim, record its cited ID, the actual field value, whether the claim follows, and any correction. Reject conclusions unsupported by the records even when the model is confident. Include one alternative explanation. The AI response is a proposal; it has no authority to approve or execute a security action.

6 Verify the corresponding operational control

Azure procedure B Report-only policy and application identity

1. Confirm the training tenant and the approved test group. Verify emergency-access accounts are outside the test scope. Do not use All users or all production resources for this exercise.
2. Microsoft Entra admin center > Protection > Conditional Access > Policies > New policy. Name it ITSEC MFA test <initials>.
3. Under Users select only the approved training group. Under Target resources select Microsoft Azure Management where available. Under Grant select Require multifactor authentication. Set Enable policy to Report-only and create it. Capture the scope, grant and mode.
4. Use a trainer-approved test sign-in. Open Monitoring & health > Sign-in logs, select the event and inspect its Report-only evaluation. Record the policy result, timestamp and test-user identifier. Do not claim that a report-only result blocked access.
5. App registrations > New registration: create a single-tenant training app itsec-app-<initials> with only trainer-required configuration. Record application/client ID and directory ID. Do not create a client secret for this exercise.
6. Open API permissions. Distinguish delegated from application permissions and record existing consent state. Do not grant tenant-wide administrator consent for unapproved scopes. Record the exact consent blocker if relevant.
7. On the assigned Azure VM or supported application resource, open Identity > System assigned. If the trainer permits enabling it, save the setting and record the principal ID; otherwise inspect the preconfigured identity.
8. On the trainer-designated target resource, inspect IAM and the identity's assigned role and scope. Use the approved workload demonstration to capture a successful resource operation or redacted access-denied result. Record the target/audience separately from resource permission; never export the token.
9. Complete the comparison with the local fixture. Remove the lab-created report-only policy and app registration after approval, and restore identity configuration only if you changed it and the trainer authorises reversal.

For every screenshot or export, record resource scope, UTC time and expected versus observed result. Redact personal data and secrets. If blocked, record the exact error, missing permission/licence, what could be inspected, and the unverified outcome. Ask the trainer to provide an authorised sandbox demonstration where the assessed ability cannot otherwise be evidenced.

7 Submit evidence and clean up

Complete evidence-template.md with baseline, changed result, AI claim review and Azure evidence or clearly labelled limitations. Keep results.json and changed-results.json with your submission. Check that your conclusion distinguishes an observed fact from a hypothesis. Remove only resources or assignments you created with trainer approval; do not delete shared training infrastructure.

Acceptance checks

- The unchanged fixture produces the exact expected report.
- The changed fixture produces the explained result and a different input hash.
- At least one AI or analyst claim is checked against a specific record and field.

- The report states simulation, Azure verified or blocked for each relevant claim.
- The output is a policy-mode comparison and workload identity diagnosis with an owner, evidence and remaining uncertainty.

Troubleshooting

- Python not found: install the trainer-approved Python distribution or use py -3 on Windows.
- File not found: open the terminal in this lab folder; check the input filename.
- JSON parse error: validate with `python3 -m json.tool changed-data.json`; use lowercase true and false in JSON.
- Baseline mismatch: restore the supplied fixture and inspect the first differing decision; never edit the expected file.
- Azure denial or missing feature: capture the exact error and scope. A blocker is a limitation, not proof of competence or deployment.
- AI invents evidence: reject the claim, cite the missing ID, and request an evidence-limited revision.

References

The Learner Guide contains the complete source register and the lecture explanations for this lab. Original examples are adapted from the supplied Azure and AI-security references. Product behaviour should be checked against current Microsoft Learn documentation before a real deployment.

Lab 05 NSG segmentation and flow diagnosis

AI for IT Security Professionals | TGS-2023039344 | v6.0

Purpose and output

Create a flow decision table with least-privilege rules. This lab supports LO2. It uses a simplified deterministic teaching model, not a trained AI system or full Azure emulator. The AI review is a separate exercise using the included prompt.

Requirements

Python 3.10 or later, a text editor and this entire folder. No packages, network access or API key are required for the baseline. On Windows use `py -3` if `python3` is unavailable. An approved AI tool is optional; the trainer can provide a review response for critique. Azure exercises require the trainer-assigned subscription, permission and feature entitlement. Record an exact blocker where unavailable; never describe an unperformed test as passed.

Folder contents

- `mock-data.json`: synthetic input with no real personal data or credentials.
- `analyse.py`: runnable analysis for this lab only.
- `expected-results.json`: expected baseline and source checksum.
- `ai-review-prompt.txt`: evidence-constrained AI review prompt.
- `evidence-template.md`: your deliverable template.
- `INSTRUCTIONS.md` and `INSTRUCTIONS.pdf`: the same procedure in two formats.

1 Prepare a working copy

Copy this whole folder to a location you can edit. Open a terminal in the copied folder. Confirm Python and inspect the data before running code.

```
python3 --version
python3 -m json.tool mock-data.json
```

Identify the record IDs and explain the meaning of each field. All values are invented classroom fixtures. Open `analyse.py` in the editor and locate the decision rule. It reads a local JSON file and writes a local report.

2 Run the baseline

```
python3 analyse.py --input mock-data.json --output results.json
```

Open `results.json`. The expected decisions in output order are: ALLOW, DENY, ALLOW. The `source_sha256` binds the report to the exact input bytes. Compare against the supplied baseline:

```
python3 -c "import json; a=json.load(open('results.json')); b=json.load(open('expected-results.json')); assert a==b; print('BASELINE PASS')"
```

A pass proves this fixture was processed as specified; it does not prove an Azure control was deployed. Copy the input checksum and decision rows into your evidence template.

3 Trace the mechanism

NSG rule precedence

Evaluate matching rules by ascending priority; the first match decides a new flow.

Contract: priority, source, destination, port, access

Accepted case: Priority 100 allows web to app TCP 443.

Counterexample: Priority 90 denies the same flow before priority 100.

Diagnosis: Inspect earlier matching rules and both subnet/NIC associations.

Evidence: Record the five-tuple, matched rule and direction.

ASG membership

An application security group groups NICs so rules can express workload roles.

Contract: source_asg=web, destination_asg=app, port=443

Accepted case: The app NIC belongs to the app group used by the rule.

Counterexample: A recreated NIC was never added to the app group.

Diagnosis: Repair membership after checking the actual NIC identity; do not broaden source to Any.

Evidence: Export NIC membership and the effective rule result.

Flow verification

A permitted packet requires matching route, filter and listening service state.

Contract: src_ip, dst_ip, dst_port, direction, protocol

Accepted case: TCP 443 is allowed and the application listens on 443.

Counterexample: NSG allows 443 but the service is stopped.

Diagnosis: Separate route, NSG, host firewall and process failures; IP flow verify alone is not an end-to-end test.

Evidence: Keep filter result plus an application connection test.

4 Change one input and test the consequence

Save a copy of mock-data.json as changed-data.json using the editor. Change flow-1 port to 8443. It must be denied. Explain the exact rule needed if the application legitimately moves to 8443; do not allow every port.

```
python3 analyse.py --input changed-data.json --output changed-results.json
```

Compare decisions and source hashes with the baseline. Identify the exact expression in analyse.py responsible for the changed behaviour. Do not overwrite expected-results.json to make a failing baseline pass. Restore the original fixture if you edited it accidentally.

5 Review AI claims against evidence

Open ai-review-prompt.txt. In an organisation-approved AI tool, submit the prompt with the synthetic input and result only. Record the tool/model name, date and prompt version if available. If no approved AI tool is available, manually draft a tentative analyst summary and label it as a human exercise.

For each returned claim, record its cited ID, the actual field value, whether the claim follows, and any correction. Reject conclusions unsupported by the records even when the model is confident. Include one alternative explanation. The AI response is a proposal; it has no authority to approve or execute a security action.

6 Verify the corresponding operational control

Azure procedure C Build and verify a segmented network

1. In rg-itsec-<initials>, create Virtual network vnet-itsec-<initials> with address space 10.10.0.0/16 in the assigned region.

2. Add subnets web (10.10.1.0/24), app (10.10.2.0/24) and data (10.10.3.0/24). Keep any service-specific reserved subnet separate.
3. Create Application security groups asg-web-<initials>, asg-app-<initials> and asg-data-<initials>. Have the trainer provision or allocate the tier VMs in this new VNet and the corresponding web/app/data subnets. Add only those NICs to the matching ASG; all NICs in an ASG must share a VNet. Record NIC and subnet IDs. If no suitable VM is allocated, record the live flow test as unperformed. Empty ASGs cannot demonstrate a working traffic path.
4. Create nsg-app-<initials> and associate it with the app subnet. In Inbound security rules add priority 100, TCP, source ASG web, destination ASG app, destination port 443, Allow. Name it Allow-Web-App-443.
5. Add priority 200, source VirtualNetwork, destination Any, any port/protocol, Deny for the intended isolation exercise only. This explicit restriction prevents Azure's default AllowVNetInBound from being mistaken for deny-all. Confirm no shared services depend on this subnet before saving.
6. Create the data-tier NSG and an inbound rule priority 100 allowing TCP 1433 from ASG app to ASG data, followed by the agreed explicit deny policy. Associate it with the data subnet. Record the rule JSON or screenshots.
7. Open Network Watcher > IP flow verify. Select an approved VM/NIC, inbound direction and a real source/destination/port tuple. Test the intended allow and deny cases and record the matched rule. A VM/NIC must exist for this operational test; a diagram is not a substitute.
8. Inspect NIC effective security rules and routes. If the filter allows traffic but the application fails, inspect the host firewall and listening service through the trainer-approved access path. Do not broaden ports to bypass the diagnosis.
9. Compare live behaviour with the simplified Python fixture: Azure also has default rules, associations and stateful connection handling absent from the script. Retain configuration, allow/deny evidence and limitations. Remove only lab-created network resources after all later labs are complete.

For every screenshot or export, record resource scope, UTC time and expected versus observed result. Redact personal data and secrets. If blocked, record the exact error, missing permission/licence, what could be inspected, and the unverified outcome. Ask the trainer to provide an authorised sandbox demonstration where the assessed ability cannot otherwise be evidenced.

7 Submit evidence and clean up

Complete evidence-template.md with baseline, changed result, AI claim review and Azure evidence or clearly labelled limitations. Keep results.json and changed-results.json with your submission. Check that your conclusion distinguishes an observed fact from a hypothesis. Remove only resources or assignments you created with trainer approval; do not delete shared training infrastructure.

Acceptance checks

- The unchanged fixture produces the exact expected report.
- The changed fixture produces the explained result and a different input hash.
- At least one AI or analyst claim is checked against a specific record and field.
- The report states simulation, Azure verified or blocked for each relevant claim.
- The output is a flow decision table with least-privilege rules with an owner, evidence and remaining uncertainty.

Troubleshooting

- Python not found: install the trainer-approved Python distribution or use py -3 on Windows.
- File not found: open the terminal in this lab folder; check the input filename.
- JSON parse error: validate with `python3 -m json.tool changed-data.json`; use lowercase true and false in JSON.
- Baseline mismatch: restore the supplied fixture and inspect the first differing decision; never edit the expected file.
- Azure denial or missing feature: capture the exact error and scope. A blocker is a limitation, not proof of competence or deployment.
- AI invents evidence: reject the claim, cite the missing ID, and request an evidence-limited revision.

References

The Learner Guide contains the complete source register and the lecture explanations for this lab. Original examples are adapted from the supplied Azure and AI-security references. Product behaviour should be checked against current Microsoft Learn documentation before a real deployment.

Lab 06 Web edge and private access

AI for IT Security Professionals | TGS-2023039344 | v6.0

Purpose and output

Create a private-access diagnosis and traffic-layer diagram. This lab supports LO2. It uses a simplified deterministic teaching model, not a trained AI system or full Azure emulator. The AI review is a separate exercise using the included prompt.

Requirements

Python 3.10 or later, a text editor and this entire folder. No packages, network access or API key are required for the baseline. On Windows use `py -3` if `python3` is unavailable. An approved AI tool is optional; the trainer can provide a review response for critique. Azure exercises require the trainer-assigned subscription, permission and feature entitlement. Record an exact blocker where unavailable; never describe an unperformed test as passed.

Folder contents

- `mock-data.json`: synthetic input with no real personal data or credentials.
- `analyse.py`: runnable analysis for this lab only.
- `expected-results.json`: expected baseline and source checksum.
- `ai-review-prompt.txt`: evidence-constrained AI review prompt.
- `evidence-template.md`: your deliverable template.
- `INSTRUCTIONS.md` and `INSTRUCTIONS.pdf`: the same procedure in two formats.

1 Prepare a working copy

Copy this whole folder to a location you can edit. Open a terminal in the copied folder. Confirm Python and inspect the data before running code.

```
python3 --version
python3 -m json.tool mock-data.json
```

Identify the record IDs and explain the meaning of each field. All values are invented classroom fixtures. Open `analyse.py` in the editor and locate the decision rule. It reads a local JSON file and writes a local report.

2 Run the baseline

```
python3 analyse.py --input mock-data.json --output results.json
```

Open `results.json`. The expected decisions in output order are: `PRIVATE_CONFIG`, `DNS_MISMATCH`, `PUBLIC_PATH_REMAINS`. The `source_sha256` binds the report to the exact input bytes. Compare against the supplied baseline:

```
python3 -c "import json; a=json.load(open('results.json')); b=json.load(open('expected-results.json')); assert a==b; print('BASELINE PASS')"
```

A pass proves this fixture was processed as specified; it does not prove an Azure control was deployed. Copy the input checksum and decision rows into your evidence template.

3 Trace the mechanism

Firewall and WAF boundaries

Firewall controls network/application egress; WAF inspects HTTP application requests.

Contract: layer, source, destination, host, request_path

Accepted case: WAF blocks a disallowed request at the web edge.

Counterexample: WAF is expected to inspect arbitrary SSH traffic.

Diagnosis: Route non-HTTP controls to the correct network control and preserve WAF request evidence.

Evidence: Show the traffic path and which layer makes each decision.

Private endpoint resolution

Private DNS resolves a service name to the endpoint NIC private address.

Contract: fqdn, resolved_ip, endpoint_ip, public_access

Accepted case: The service resolves to 10.10.3.5 from the training VNet.

Counterexample: It resolves to a public address despite an approved endpoint.

Diagnosis: Check DNS zone linkage, resolver path and public-access configuration.

Evidence: Record DNS output and endpoint approval state; endpoint creation alone is insufficient.

Egress allowlists

The gateway authorises a destination independently of an AI agent's requested action.

Contract: tool, host, port, approved_destination

Accepted case: A report goes to reports.example.test on port 443.

Counterexample: A retrieved document asks for upload to unknown.example.test.

Diagnosis: Deny unapproved destinations and investigate the document as untrusted input.

Evidence: Log destination, policy result and correlation ID without exporting content.

4 Change one input and test the consequence

Save a copy of mock-data.json as changed-data.json using the editor. Correct storage-b dns_ip to 10.10.3.6. Its result becomes PRIVATE_CONFIG, but list the real connection and authentication tests still required.

```
python3 analyse.py --input changed-data.json --output changed-results.json
```

Compare decisions and source hashes with the baseline. Identify the exact expression in analyse.py responsible for the changed behaviour. Do not overwrite expected-results.json to make a failing baseline pass. Restore the original fixture if you edited it accidentally.

5 Review AI claims against evidence

Open ai-review-prompt.txt. In an organisation-approved AI tool, submit the prompt with the synthetic input and result only. Record the tool/model name, date and prompt version if available. If no approved AI tool is available, manually draft a tentative analyst summary and label it as a human exercise.

For each returned claim, record its cited ID, the actual field value, whether the claim follows, and any correction. Reject conclusions unsupported by the records even when the model is confident. Include one alternative explanation. The AI response is a proposal; it has no authority to approve or execute a security action.

6 Verify the corresponding operational control

Azure procedure D Inspect web edge and private service access

1. Open the trainer-provisioned Azure Firewall and its Firewall Policy. Record the protected subnet, route path and one approved rule collection. If no firewall is provisioned, review the supplied trainer architecture and record deployment as unverified; do not create a billable appliance without allocation.
2. Open the assigned Web Application Firewall policy. Record its association to the web edge, its mode and one applicable managed/custom rule. Explain that Detection logs and Prevention enforcement are different states.
3. For the allocated storage account, open Networking > Private endpoint connections. Create an endpoint only where the trainer has assigned quota and cost. Select the correct subscription, rg-itsec-<initials>, storage target and required subresource, such as blob.
4. Select the approved VNet/subnet and integrate with the appropriate private DNS zone. Review the target resource and create. Record the endpoint NIC private IP and connection approval state.
5. Open the private DNS zone and its Virtual network links. Confirm the training VNet is linked. From a trainer-approved VM in that VNet, run nslookup <storage-account>.blob.core.windows.net and record the answer chain and final address.
6. Compare the final address with the endpoint NIC address. If it resolves publicly, inspect zone linkage and the resolver path. Do not change public DNS arbitrarily.
7. Review the storage account public network access setting independently. If the exercise requires disabling it, confirm the private path works and get the trainer's go-ahead before changing it.
8. Record one authorised private data operation and a disallowed public-path test where the sandbox supports both. Private DNS alone does not prove authentication or public denial.
9. Save firewall/WAF mode, endpoint approval, DNS and access evidence. Remove only the allocated endpoint and its lab-owned DNS resources after the trainer approves cleanup.

For every screenshot or export, record resource scope, UTC time and expected versus observed result. Redact personal data and secrets. If blocked, record the exact error, missing permission/licence, what could be inspected, and the unverified outcome. Ask the trainer to provide an authorised sandbox demonstration where the assessed ability cannot otherwise be evidenced.

7 Submit evidence and clean up

Complete evidence-template.md with baseline, changed result, AI claim review and Azure evidence or clearly labelled limitations. Keep results.json and changed-results.json with your submission. Check that your conclusion distinguishes an observed fact from a hypothesis. Remove only resources or assignments you created with trainer approval; do not delete shared training infrastructure.

Acceptance checks

- The unchanged fixture produces the exact expected report.
- The changed fixture produces the explained result and a different input hash.
- At least one AI or analyst claim is checked against a specific record and field.
- The report states simulation, Azure verified or blocked for each relevant claim.

- The output is a private-access diagnosis and traffic-layer diagram with an owner, evidence and remaining uncertainty.

Troubleshooting

- Python not found: install the trainer-approved Python distribution or use py -3 on Windows.
- File not found: open the terminal in this lab folder; check the input filename.
- JSON parse error: validate with `python3 -m json.tool changed-data.json`; use lowercase true and false in JSON.
- Baseline mismatch: restore the supplied fixture and inspect the first differing decision; never edit the expected file.
- Azure denial or missing feature: capture the exact error and scope. A blocker is a limitation, not proof of competence or deployment.
- AI invents evidence: reject the claim, cite the missing ID, and request an evidence-limited revision.

References

The Learner Guide contains the complete source register and the lecture explanations for this lab. Original examples are adapted from the supplied Azure and AI-security references. Product behaviour should be checked against current Microsoft Learn documentation before a real deployment.

Lab 07 VM administration and encryption

AI for IT Security Professionals | TGS-2023039344 | v6.0

Purpose and output

Create a VM control evidence matrix. This lab supports LO2. It uses a simplified deterministic teaching model, not a trained AI system or full Azure emulator. The AI review is a separate exercise using the included prompt.

Requirements

Python 3.10 or later, a text editor and this entire folder. No packages, network access or API key are required for the baseline. On Windows use `py -3` if `python3` is unavailable. An approved AI tool is optional; the trainer can provide a review response for critique. Azure exercises require the trainer-assigned subscription, permission and feature entitlement. Record an exact blocker where unavailable; never describe an unperformed test as passed.

Folder contents

- `mock-data.json`: synthetic input with no real personal data or credentials.
- `analyse.py`: runnable analysis for this lab only.
- `expected-results.json`: expected baseline and source checksum.
- `ai-review-prompt.txt`: evidence-constrained AI review prompt.
- `evidence-template.md`: your deliverable template.
- `INSTRUCTIONS.md` and `INSTRUCTIONS.pdf`: the same procedure in two formats.

1 Prepare a working copy

Copy this whole folder to a location you can edit. Open a terminal in the copied folder. Confirm Python and inspect the data before running code.

```
python3 --version
python3 -m json.tool mock-data.json
```

Identify the record IDs and explain the meaning of each field. All values are invented classroom fixtures. Open `analyse.py` in the editor and locate the decision rule. It reads a local JSON file and writes a local report.

2 Run the baseline

```
python3 analyse.py --input mock-data.json --output results.json
```

Open `results.json`. The expected decisions in output order are: `BASELINE_OK`, `PUBLIC_ADMIN`, `LONG_WINDOW`, `ENCRYPTION_REVIEW`. The `source_sha256` binds the report to the exact input bytes. Compare against the supplied baseline:

```
python3 -c "import json; a=json.load(open('results.json')); b=json.load(open('expected-results.json')); assert a==b; print('BASELINE PASS')"
```

A pass proves this fixture was processed as specified; it does not prove an Azure control was deployed. Copy the input checksum and decision rows into your evidence template.

3 Trace the mechanism

Bastion access path

A managed jump path can remove direct public administrative exposure on a VM.

Contract: `vm_public_ip`, `admin_port`, `bastion_path`

Accepted case: The VM has no public IP and uses the approved Bastion path.

Counterexample: TCP 3389 is open from the internet alongside Bastion.

Diagnosis: Remove the direct exposure after validating the managed access path and recovery plan.

Evidence: Capture NIC public IP state and effective inbound rules.

Just in time administration

Time-limited access narrows source and duration for administrative ports.

Contract: source_cidr, port, requested_minutes, expiry

Accepted case: One approved source opens port 22 for 30 minutes.

Counterexample: 0.0.0.0/0 is allowed without an expiry.

Diagnosis: Close the broad rule and verify access expires; approval is distinct from successful use.

Evidence: Keep request, approval, start, expiry and post-expiry denial evidence.

Encryption and identity

Encryption protects stored bytes while identity controls authorised plaintext access.

Contract: encryption_state, key_reference, principal

Accepted case: Encrypted disks remain protected by scoped operator access.

Counterexample: A broad identity reads data legitimately despite encryption.

Diagnosis: Reduce identity permissions; encryption cannot compensate for authorised over-access.

Evidence: Separate encryption configuration evidence from access test results.

4 Change one input and test the consequence

Save a copy of mock-data.json as changed-data.json using the editor. Set vm-b public_admin to false. LONG_WINDOW must remain; closing one exposure does not satisfy all controls.

```
python3 analyse.py --input changed-data.json --output changed-results.json
```

Compare decisions and source hashes with the baseline. Identify the exact expression in analyse.py responsible for the changed behaviour. Do not overwrite expected-results.json to make a failing baseline pass. Restore the original fixture if you edited it accidentally.

5 Review AI claims against evidence

Open ai-review-prompt.txt. In an organisation-approved AI tool, submit the prompt with the synthetic input and result only. Record the tool/model name, date and prompt version if available. If no approved AI tool is available, manually draft a tentative analyst summary and label it as a human exercise.

For each returned claim, record its cited ID, the actual field value, whether the claim follows, and any correction. Reject conclusions unsupported by the records even when the model is confident. Include one alternative explanation. The AI response is a proposal; it has no authority to approve or execute a security action.

6 Verify the corresponding operational control

Azure procedure E Verify VM administration and encryption

1. Open the assigned VM > Networking. Record its NIC, subnet and whether a public IP is attached. Inspect effective inbound rules for ports 22 and 3389.

2. Review the provisioned Bastion path and its VNet/subnet placement. Connect to the training VM using the approved identity if access is allocated. Do not record credentials.
3. Identify any direct public administrative rule that remains. Propose its removal, verify an alternative access/recovery route with the trainer, and change only an allocated lab rule.
4. In Defender for Cloud, locate the VM's Just-in-Time access configuration where the plan supports it. Request a trainer-approved short access window from a single approved source. Record port, source and expiry.
5. Test the access while active. After expiry, test a new connection and record denial; an existing connection can behave differently from a new one. Capture the policy and actual result separately.
6. VM > Disks: inspect encryption at rest and any encryption-set/key reference. Determine which layer is being shown rather than assuming all encryption options are equivalent.
7. Explain why a legitimately authorised broad identity can still access plaintext despite encryption. Record separate identity and encryption evidence.
8. Restore lab-only access changes and verify that no temporary administrative rule remains. Record feature/licence blockers without claiming a successful deployment.

For every screenshot or export, record resource scope, UTC time and expected versus observed result. Redact personal data and secrets. If blocked, record the exact error, missing permission/licence, what could be inspected, and the unverified outcome. Ask the trainer to provide an authorised sandbox demonstration where the assessed ability cannot otherwise be evidenced.

7 Submit evidence and clean up

Complete evidence-template.md with baseline, changed result, AI claim review and Azure evidence or clearly labelled limitations. Keep results.json and changed-results.json with your submission. Check that your conclusion distinguishes an observed fact from a hypothesis. Remove only resources or assignments you created with trainer approval; do not delete shared training infrastructure.

Acceptance checks

- The unchanged fixture produces the exact expected report.
- The changed fixture produces the explained result and a different input hash.
- At least one AI or analyst claim is checked against a specific record and field.
- The report states simulation, Azure verified or blocked for each relevant claim.
- The output is a VM control evidence matrix with an owner, evidence and remaining uncertainty.

Troubleshooting

- Python not found: install the trainer-approved Python distribution or use py -3 on Windows.
- File not found: open the terminal in this lab folder; check the input filename.
- JSON parse error: validate with python3 -m json.tool changed-data.json; use lowercase true and false in JSON.
- Baseline mismatch: restore the supplied fixture and inspect the first differing decision; never edit the expected file.
- Azure denial or missing feature: capture the exact error and scope. A blocker is a limitation, not proof of competence or deployment.

- AI invents evidence: reject the claim, cite the missing ID, and request an evidence-limited revision.

References

The Learner Guide contains the complete source register and the lecture explanations for this lab. Original examples are adapted from the supplied Azure and AI-security references. Product behaviour should be checked against current Microsoft Learn documentation before a real deployment.

Lab 08 Storage SQL and secret protection

AI for IT Security Professionals | TGS-2023039344 | v6.0

Purpose and output

Create a data-access and recovery exception report. This lab supports LO2. It uses a simplified deterministic teaching model, not a trained AI system or full Azure emulator. The AI review is a separate exercise using the included prompt.

Requirements

Python 3.10 or later, a text editor and this entire folder. No packages, network access or API key are required for the baseline. On Windows use `py -3` if `python3` is unavailable. An approved AI tool is optional; the trainer can provide a review response for critique. Azure exercises require the trainer-assigned subscription, permission and feature entitlement. Record an exact blocker where unavailable; never describe an unperformed test as passed.

Folder contents

- `mock-data.json`: synthetic input with no real personal data or credentials.
- `analyse.py`: runnable analysis for this lab only.
- `expected-results.json`: expected baseline and source checksum.
- `ai-review-prompt.txt`: evidence-constrained AI review prompt.
- `evidence-template.md`: your deliverable template.
- `INSTRUCTIONS.md` and `INSTRUCTIONS.pdf`: the same procedure in two formats.

1 Prepare a working copy

Copy this whole folder to a location you can edit. Open a terminal in the copied folder. Confirm Python and inspect the data before running code.

```
python3 --version
python3 -m json.tool mock-data.json
```

Identify the record IDs and explain the meaning of each field. All values are invented classroom fixtures. Open `analyse.py` in the editor and locate the decision rule. It reads a local JSON file and writes a local report.

2 Run the baseline

```
python3 analyse.py --input mock-data.json --output results.json
```

Open `results.json`. The expected decisions in output order are: `BASELINE_OK`, `ACCESS_REVIEW`, `SECRET_EXPOSURE`, `RPO_BREACH`. The `source_sha256` binds the report to the exact input bytes. Compare against the supplied baseline:

```
python3 -c "import json; a=json.load(open('results.json')); b=json.load(open('expected-results.json')); assert a==b; print('BASELINE PASS')"
```

A pass proves this fixture was processed as specified; it does not prove an Azure control was deployed. Copy the input checksum and decision rows into your evidence template.

3 Trace the mechanism

Storage access boundaries

Data-plane roles, shared keys, SAS and public access create distinct access paths.

Contract: `allow_public`, `allow_shared_key`, `sas_expiry`, `data_role`

Accepted case: An identity receives scoped blob data access with public access disabled.

Counterexample: An account key enables broad access outside the intended identity scope.

Diagnosis: Inventory every enabled access path before calling a storage account private.

Evidence: Capture role scope, anonymous denial and expiry of temporary grants.

SQL and secret protection

A database authenticates a principal and authorises its operations; a vault limits secret retrieval.

Contract: sql_principal, database_role, vault_role, secret_version

Accepted case: A read-only database user and scoped secret reader serve a reporting workload.

Counterexample: A shared administrator secret is pasted into a notebook.

Diagnosis: Use an approved identity path and rotate any exposed live secret; keep values out of screenshots.

Evidence: Record role names and secret metadata, never the secret value.

Retention and recovery

Recovery points and restore tests establish recoverability independently of encryption.

Contract: backup_time, restore_time, rpo_minutes, rto_minutes

Accepted case: A 15-minute data gap meets a 30-minute RPO.

Counterexample: A backup exists but the restore fails due to missing key access.

Diagnosis: Test restoration in an isolated target and include key availability in the recovery design.

Evidence: Record measured data loss and elapsed restoration time.

4 Change one input and test the consequence

Save a copy of mock-data.json as changed-data.json using the editor. Set data-c secret_in_code to false. RPO_BREACH must remain; access hygiene and recovery objectives are separate.

```
python3 analyse.py --input changed-data.json --output changed-results.json
```

Compare decisions and source hashes with the baseline. Identify the exact expression in analyse.py responsible for the changed behaviour. Do not overwrite expected-results.json to make a failing baseline pass. Restore the original fixture if you edited it accidentally.

5 Review AI claims against evidence

Open ai-review-prompt.txt. In an organisation-approved AI tool, submit the prompt with the synthetic input and result only. Record the tool/model name, date and prompt version if available. If no approved AI tool is available, manually draft a tentative analyst summary and label it as a human exercise.

For each returned claim, record its cited ID, the actual field value, whether the claim follows, and any correction. Reject conclusions unsupported by the records even when the model is confident. Include one alternative explanation. The AI response is a proposal; it has no authority to approve or execute a security action.

6 Verify the corresponding operational control

Azure procedure F Storage SQL and vault controls

1. Open the assigned storage account > Configuration and Networking. Record anonymous/public and shared-key settings. Inspect data-plane role assignments separately from management roles.

2. Use the trainer-approved test identity to demonstrate one allowed data operation and one operation outside its intended scope. Do not list or copy account keys merely to obtain access.
3. Review a synthetic or trainer-approved SAS example. Identify service/resource scope, permissions and expiry. Do not place a live SAS URL in the evidence or an AI prompt.
4. Open the assigned Key Vault > Access configuration and identify the permission model. For Azure RBAC inspect Access control (IAM); for the legacy model inspect Access policies. Record the test identity's actual secret-access scope under the selected model. Create a synthetic training secret only if allocated; use a value such as TRAINING-ONLY-NOT-A-CREDENTIAL and never reuse a real secret.
5. Capture secret name/version metadata and an authorised or denied test, not the value. Remove the synthetic secret under trainer supervision after evidence capture.
6. Open the assigned Azure SQL server/database > Security. Review authentication configuration, network access and auditing. In an approved query session inspect the assigned database role without granting administrator access.
7. Demonstrate a permitted read and a denied ungranted operation with the test principal where allocated. Record the precise database scope and identity used.
8. Review the recovery plan. With an allocated test target, restore a training backup and measure data loss and elapsed time; otherwise explicitly mark the restore unperformed. Compare to the RPO/RTO requirement.
9. Submit the access matrix, secret-handling review and recovery evidence; clean up only the trainer-approved synthetic assets.

For every screenshot or export, record resource scope, UTC time and expected versus observed result. Redact personal data and secrets. If blocked, record the exact error, missing permission/licence, what could be inspected, and the unverified outcome. Ask the trainer to provide an authorised sandbox demonstration where the assessed ability cannot otherwise be evidenced.

7 Submit evidence and clean up

Complete evidence-template.md with baseline, changed result, AI claim review and Azure evidence or clearly labelled limitations. Keep results.json and changed-results.json with your submission. Check that your conclusion distinguishes an observed fact from a hypothesis. Remove only resources or assignments you created with trainer approval; do not delete shared training infrastructure.

Acceptance checks

- The unchanged fixture produces the exact expected report.
- The changed fixture produces the explained result and a different input hash.
- At least one AI or analyst claim is checked against a specific record and field.
- The report states simulation, Azure verified or blocked for each relevant claim.
- The output is a data-access and recovery exception report with an owner, evidence and remaining uncertainty.

Troubleshooting

- Python not found: install the trainer-approved Python distribution or use py -3 on Windows.
- File not found: open the terminal in this lab folder; check the input filename.

- JSON parse error: validate with `python3 -m json.tool changed-data.json`; use lowercase true and false in JSON.
- Baseline mismatch: restore the supplied fixture and inspect the first differing decision; never edit the expected file.
- Azure denial or missing feature: capture the exact error and scope. A blocker is a limitation, not proof of competence or deployment.
- AI invents evidence: reject the claim, cite the missing ID, and request an evidence-limited revision.

References

The Learner Guide contains the complete source register and the lecture explanations for this lab. Original examples are adapted from the supplied Azure and AI-security references. Product behaviour should be checked against current Microsoft Learn documentation before a real deployment.

Lab 09 Policy and security posture

AI for IT Security Professionals | TGS-2023039344 | v6.0

Purpose and output

Create a policy finding register with accountable owners. This lab supports LO1. It uses a simplified deterministic teaching model, not a trained AI system or full Azure emulator. The AI review is a separate exercise using the included prompt.

Requirements

Python 3.10 or later, a text editor and this entire folder. No packages, network access or API key are required for the baseline. On Windows use `py -3` if `python3` is unavailable. An approved AI tool is optional; the trainer can provide a review response for critique. Azure exercises require the trainer-assigned subscription, permission and feature entitlement. Record an exact blocker where unavailable; never describe an unperformed test as passed.

Folder contents

- `mock-data.json`: synthetic input with no real personal data or credentials.
- `analyse.py`: runnable analysis for this lab only.
- `expected-results.json`: expected baseline and source checksum.
- `ai-review-prompt.txt`: evidence-constrained AI review prompt.
- `evidence-template.md`: your deliverable template.
- `INSTRUCTIONS.md` and `INSTRUCTIONS.pdf`: the same procedure in two formats.

1 Prepare a working copy

Copy this whole folder to a location you can edit. Open a terminal in the copied folder. Confirm Python and inspect the data before running code.

```
python3 --version
python3 -m json.tool mock-data.json
```

Identify the record IDs and explain the meaning of each field. All values are invented classroom fixtures. Open `analyse.py` in the editor and locate the decision rule. It reads a local JSON file and writes a local report.

2 Run the baseline

```
python3 analyse.py --input mock-data.json --output results.json
```

Open `results.json`. The expected decisions in output order are: `AUDIT_FINDING`, `WOULD_DENY_REQUEST`, `EXPIRED_EXCEPTION`. The `source_sha256` binds the report to the exact input bytes. Compare against the supplied baseline:

```
python3 -c "import json; a=json.load(open('results.json')); b=json.load(open('expected-results.json')); assert a==b; print('BASELINE PASS')"
```

A pass proves this fixture was processed as specified; it does not prove an Azure control was deployed. Copy the input checksum and decision rows into your evidence template.

3 Trace the mechanism

Policy effect semantics

Audit records noncompliance; deny prevents supported noncompliant resource requests.

Contract: effect, assignment_scope, resource_type

Accepted case: Audit flags public exposure for review.

Counterexample: An audit-only assignment is described as blocking deployment.

Diagnosis: Choose the intended effect, examine exemptions and re-evaluate actual resource state.

Evidence: Capture effect, scope, exemption and compliance timestamp.

Posture prioritisation

A posture recommendation becomes actionable when tied to affected assets and validated exposure.

Contract: recommendation_id, resource_id, severity, owner

Accepted case: A critical recommendation affects the internet-facing payment VM.

Counterexample: A stale recommendation references a removed resource.

Diagnosis: Verify existence and exposure before scheduling remediation; score changes are not risk proof.

Evidence: Record resource state, owner, remediation ticket and retest.

Exception lifecycle

A risk exception is bounded by an owner, compensating control and expiry.

Contract: exception_id, expires_utc, owner, compensating_control

Accepted case: A temporary exception expires after a planned migration.

Counterexample: An indefinite exception suppresses a recurring critical finding.

Diagnosis: Escalate overdue exceptions and restore normal evaluation after expiry.

Evidence: List expired exceptions separately from compliant controls.

4 Change one input and test the consequence

Save a copy of mock-data.json as changed-data.json using the editor. Change pol-a effect to deny. Explain that the model describes a new request, not automatic remediation of an existing resource.

```
python3 analyse.py --input changed-data.json --output changed-results.json
```

Compare decisions and source hashes with the baseline. Identify the exact expression in analyse.py responsible for the changed behaviour. Do not overwrite expected-results.json to make a failing baseline pass. Restore the original fixture if you edited it accidentally.

5 Review AI claims against evidence

Open ai-review-prompt.txt. In an organisation-approved AI tool, submit the prompt with the synthetic input and result only. Record the tool/model name, date and prompt version if available. If no approved AI tool is available, manually draft a tentative analyst summary and label it as a human exercise.

For each returned claim, record its cited ID, the actual field value, whether the claim follows, and any correction. Reject conclusions unsupported by the records even when the model is confident. Include one alternative explanation. The AI response is a proposal; it has no authority to approve or execute a security action.

6 Verify the corresponding operational control

Azure procedure G Policy and Defender posture

1. Azure Portal > Policy > Definitions: locate the trainer-approved audit definition for the chosen control. Open its rule and identify the effect and evaluated resource fields.
2. Assign it only to rg-itsec-<initials>. Name the assignment clearly and record parameters, exclusions and any managed identity requirement. Avoid a production Deny assignment for this exercise.
3. Open Compliance and record the evaluation time and affected resource. Policy evaluation is asynchronous; if no current result is available, state that it is pending and retain the assignment evidence.
4. Compare Audit with a trainer-demonstrated Deny request. Explain why the latter describes enforcement on a supported request and does not prove existing resources were repaired.
5. Defender for Cloud > Recommendations: select a finding for the allocated resource. Verify the resource still exists and record severity, exposed surface and remediation guidance.
6. Review secure score, regulatory-compliance view and enabled plan status. Record plan/licence limitations and avoid presenting a numerical score as proof of compliance.
7. Create a remediation record with owner, priority, required approval and retest. Document any exception's compensating control and expiry.
8. Retest the changed control only after an authorised sandbox change. Remove the lab policy assignment when the trainer confirms cleanup; preserve before/after evidence.

For every screenshot or export, record resource scope, UTC time and expected versus observed result. Redact personal data and secrets. If blocked, record the exact error, missing permission/licence, what could be inspected, and the unverified outcome. Ask the trainer to provide an authorised sandbox demonstration where the assessed ability cannot otherwise be evidenced.

7 Submit evidence and clean up

Complete evidence-template.md with baseline, changed result, AI claim review and Azure evidence or clearly labelled limitations. Keep results.json and changed-results.json with your submission. Check that your conclusion distinguishes an observed fact from a hypothesis. Remove only resources or assignments you created with trainer approval; do not delete shared training infrastructure.

Acceptance checks

- The unchanged fixture produces the exact expected report.
- The changed fixture produces the explained result and a different input hash.
- At least one AI or analyst claim is checked against a specific record and field.
- The report states simulation, Azure verified or blocked for each relevant claim.
- The output is a policy finding register with accountable owners with an owner, evidence and remaining uncertainty.

Troubleshooting

- Python not found: install the trainer-approved Python distribution or use py -3 on Windows.
- File not found: open the terminal in this lab folder; check the input filename.

- JSON parse error: validate with `python3 -m json.tool changed-data.json`; use lowercase true and false in JSON.
- Baseline mismatch: restore the supplied fixture and inspect the first differing decision; never edit the expected file.
- Azure denial or missing feature: capture the exact error and scope. A blocker is a limitation, not proof of competence or deployment.
- AI invents evidence: reject the claim, cite the missing ID, and request an evidence-limited revision.

References

The Learner Guide contains the complete source register and the lecture explanations for this lab. Original examples are adapted from the supplied Azure and AI-security references. Product behaviour should be checked against current Microsoft Learn documentation before a real deployment.

Lab 10 Security logs and detection metrics

AI for IT Security Professionals | TGS-2023039344 | v6.0

Purpose and output

Create a time-window detection output and tuning note. This lab supports LO4. It uses a simplified deterministic teaching model, not a trained AI system or full Azure emulator. The AI review is a separate exercise using the included prompt.

Requirements

Python 3.10 or later, a text editor and this entire folder. No packages, network access or API key are required for the baseline. On Windows use `py -3` if `python3` is unavailable. An approved AI tool is optional; the trainer can provide a review response for critique. Azure exercises require the trainer-assigned subscription, permission and feature entitlement. Record an exact blocker where unavailable; never describe an unperformed test as passed.

Folder contents

- `mock-data.json`: synthetic input with no real personal data or credentials.
- `analyse.py`: runnable analysis for this lab only.
- `expected-results.json`: expected baseline and source checksum.
- `ai-review-prompt.txt`: evidence-constrained AI review prompt.
- `evidence-template.md`: your deliverable template.
- `INSTRUCTIONS.md` and `INSTRUCTIONS.pdf`: the same procedure in two formats.

1 Prepare a working copy

Copy this whole folder to a location you can edit. Open a terminal in the copied folder. Confirm Python and inspect the data before running code.

```
python3 --version
python3 -m json.tool mock-data.json
```

Identify the record IDs and explain the meaning of each field. All values are invented classroom fixtures. Open `analyse.py` in the editor and locate the decision rule. It reads a local JSON file and writes a local report.

2 Run the baseline

```
python3 analyse.py --input mock-data.json --output results.json
```

Open `results.json`. The expected decisions in output order are: `ALERT`, `NO_ALERT`. The `source_sha256` binds the report to the exact input bytes. Compare against the supplied baseline:

```
python3 -c "import json; a=json.load(open('results.json')); b=json.load(open('expected-results.json')); assert a==b; print('BASELINE PASS')"
```

A pass proves this fixture was processed as specified; it does not prove an Azure control was deployed. Copy the input checksum and decision rows into your evidence template.

3 Trace the mechanism

Log schema and time

Normalise event time and actor identifiers before correlating records.

Contract: `event_id`, `event_time_utc`, `user_id`, `operation`, `result`

Accepted case: Two sources use UTC and the same stable actor ID.

Counterexample: Local time is interpreted as UTC and changes event ordering.

Diagnosis: Retain raw time and normalisation rules; distinguish ingestion delay from event time.

Evidence: Record timezone conversion and the event-to-ingestion delay.

Detection threshold

Group failures by actor and time window; compare the count to a defined threshold.

Contract: `window_minutes=5, failed_count>=5`

Accepted case: Six failures for user_004 in five minutes trigger review.

Counterexample: Six failures spread across six hours do not meet the rule.

Diagnosis: Use time-bounded grouping and validate service-account baselines to reduce noise.

Evidence: Retain the contributing event IDs and precise window boundaries.

Detection metrics

Compare detector output to labelled cases to measure precision and recall.

Contract: $\text{precision} = \text{TP} / (\text{TP} + \text{FP})$; $\text{recall} = \text{TP} / (\text{TP} + \text{FN})$

Accepted case: TP=8, FP=2, FN=4 gives 80% precision and 66.7% recall.

Counterexample: Reporting accuracy alone hides missed incidents in an imbalanced set.

Diagnosis: Review false negatives and tune using a held-out dataset; avoid testing on tuning data.

Evidence: Save confusion counts, threshold and dataset version.

4 Change one input and test the consequence

Save a copy of mock-data.json as changed-data.json using the editor. Move e5 to minute 10. The five-minute peak becomes 4 and the alert disappears. Explain the blind spot and propose a separately tested longer-window detector.

```
python3 analyse.py --input changed-data.json --output changed-results.json
```

Compare decisions and source hashes with the baseline. Identify the exact expression in analyse.py responsible for the changed behaviour. Do not overwrite expected-results.json to make a failing baseline pass. Restore the original fixture if you edited it accidentally.

5 Review AI claims against evidence

Open ai-review-prompt.txt. In an organisation-approved AI tool, submit the prompt with the synthetic input and result only. Record the tool/model name, date and prompt version if available. If no approved AI tool is available, manually draft a tentative analyst summary and label it as a human exercise.

For each returned claim, record its cited ID, the actual field value, whether the claim follows, and any correction. Reject conclusions unsupported by the records even when the model is confident. Include one alternative explanation. The AI response is a proposal; it has no authority to approve or execute a security action.

6 Verify the corresponding operational control

Azure procedure H Log Analytics and Sentinel detection

1. Azure Portal > Log Analytics workspaces > Create: use the assigned subscription, rg-itsec-<initials>, approved region and a unique workspace name. Do this only within the trainer's cost allocation.

2. Open Microsoft Sentinel in the supported portal used by the training tenant. Add the allocated workspace where permitted. Record the workspace/resource ID and onboarding result.
3. Open Content hub and locate the Azure Activity solution if required by the tenant. Install or review the approved solution. Open its data connector instructions and use the supported policy-based connection method for the assigned subscription only.
4. Have the authorised trainer configure the required Azure Activity data-collection policy/rule if your role cannot do so. Record the assignment and workspace destination. Do not claim ingestion until an event is observed.
5. In Logs run AzureActivity | take 10. Record available fields and TimeGenerated. If the table is absent, inspect the connector, data destination and permissions; document the blocker.
6. Run AzureActivity | summarize Events=count() by bin(TimeGenerated, 5m), OperationNameValue. Record the time filter and output. This is an activity-volume query, not the synthetic failed-sign-in detector; do not label its count as failed sign-ins.
7. Create or review a scheduled analytics rule using a trainer-approved query and actual available schema. Record frequency, lookback, threshold, entity mapping and incident settings. Keep automated response disabled until separately approved.
8. Inspect a training incident's alerts and entities, then follow Lab 12 to build the timeline and Lab 16 to review response and recovery. Record the underlying events rather than relying only on an AI summary.
9. Review automation rules and a Logic Apps playbook. Identify the trigger identity, allowed actions and approval gate. Do not execute production containment. Disable/remove lab-created rules and workspaces only after the trainer approves cleanup.

For every screenshot or export, record resource scope, UTC time and expected versus observed result. Redact personal data and secrets. If blocked, record the exact error, missing permission/licence, what could be inspected, and the unverified outcome. Ask the trainer to provide an authorised sandbox demonstration where the assessed ability cannot otherwise be evidenced.

7 Submit evidence and clean up

Complete evidence-template.md with baseline, changed result, AI claim review and Azure evidence or clearly labelled limitations. Keep results.json and changed-results.json with your submission. Check that your conclusion distinguishes an observed fact from a hypothesis. Remove only resources or assignments you created with trainer approval; do not delete shared training infrastructure.

Acceptance checks

- The unchanged fixture produces the exact expected report.
- The changed fixture produces the explained result and a different input hash.
- At least one AI or analyst claim is checked against a specific record and field.
- The report states simulation, Azure verified or blocked for each relevant claim.
- The output is a time-window detection output and tuning note with an owner, evidence and remaining uncertainty.

Troubleshooting

- Python not found: install the trainer-approved Python distribution or use py -3 on Windows.
- File not found: open the terminal in this lab folder; check the input filename.

- JSON parse error: validate with `python3 -m json.tool changed-data.json`; use lowercase true and false in JSON.
- Baseline mismatch: restore the supplied fixture and inspect the first differing decision; never edit the expected file.
- Azure denial or missing feature: capture the exact error and scope. A blocker is a limitation, not proof of competence or deployment.
- AI invents evidence: reject the claim, cite the missing ID, and request an evidence-limited revision.

References

The Learner Guide contains the complete source register and the lecture explanations for this lab. Original examples are adapted from the supplied Azure and AI-security references. Product behaviour should be checked against current Microsoft Learn documentation before a real deployment.

Lab 11 Access rights troubleshooting

AI for IT Security Professionals | TGS-2023039344 | v6.0

Purpose and output

Create a access diagnosis and review decisions. This lab supports LO3. It uses a simplified deterministic teaching model, not a trained AI system or full Azure emulator. The AI review is a separate exercise using the included prompt.

Requirements

Python 3.10 or later, a text editor and this entire folder. No packages, network access or API key are required for the baseline. On Windows use `py -3` if `python3` is unavailable. An approved AI tool is optional; the trainer can provide a review response for critique. Azure exercises require the trainer-assigned subscription, permission and feature entitlement. Record an exact blocker where unavailable; never describe an unperformed test as passed.

Folder contents

- `mock-data.json`: synthetic input with no real personal data or credentials.
- `analyse.py`: runnable analysis for this lab only.
- `expected-results.json`: expected baseline and source checksum.
- `ai-review-prompt.txt`: evidence-constrained AI review prompt.
- `evidence-template.md`: your deliverable template.
- `INSTRUCTIONS.md` and `INSTRUCTIONS.pdf`: the same procedure in two formats.

1 Prepare a working copy

Copy this whole folder to a location you can edit. Open a terminal in the copied folder. Confirm Python and inspect the data before running code.

```
python3 --version
python3 -m json.tool mock-data.json
```

Identify the record IDs and explain the meaning of each field. All values are invented classroom fixtures. Open `analyse.py` in the editor and locate the decision rule. It reads a local JSON file and writes a local report.

2 Run the baseline

```
python3 analyse.py --input mock-data.json --output results.json
```

Open `results.json`. The expected decisions in output order are: `ALLOW`, `MISSING_ACTION`, `SELF_APPROVAL`. The `source_sha256` binds the report to the exact input bytes. Compare against the supplied baseline:

```
python3 -c "import json; a=json.load(open('results.json')); b=json.load(open('expected-results.json')); assert a==b; print('BASELINE PASS')"
```

A pass proves this fixture was processed as specified; it does not prove an Azure control was deployed. Copy the input checksum and decision rows into your evidence template.

3 Trace the mechanism

Least privilege troubleshooting

Diagnose actor, scope, action and policy before granting broader access.

Contract: `actor_id`, `target_scope`, `attempted_action`, `error_code`

Accepted case: The failing action is absent at the intended resource scope.

Counterexample: Owner is granted without identifying the failing action.

Diagnosis: Add only a justified permission through approval and repeat a negative test outside scope.

Evidence: Keep the original error, policy evaluation and post-fix result.

Access review decisions

Compare actual grants to job need and make an accountable retain/revoke decision.

Contract: principal, grant, business_need, reviewer, decision

Accepted case: An inactive contractor's unneeded grant is revoked.

Counterexample: A reviewer approves every grant without evidence of need.

Diagnosis: Investigate stale accounts and verify revocation across group and direct assignments.

Evidence: Record reviewer reasoning and a subsequent denied access attempt.

Separation of duties

A requester cannot approve their own privileged change when the process requires independent review.

Contract: requester_id, approver_id, requested_action

Accepted case: user_004 requests and owner_002 approves.

Counterexample: The requester and approver are the same principal.

Diagnosis: Reject self-approval and route to an authorised independent owner.

Evidence: Capture both identities and the immutable request identifier.

4 Change one input and test the consequence

Save a copy of mock-data.json as changed-data.json using the editor. Add write to req-b grants and leave req-c unchanged. Only req-b should become ALLOW. Explain why role permission does not replace independent approval.

```
python3 analyse.py --input changed-data.json --output changed-results.json
```

Compare decisions and source hashes with the baseline. Identify the exact expression in analyse.py responsible for the changed behaviour. Do not overwrite expected-results.json to make a failing baseline pass. Restore the original fixture if you edited it accidentally.

5 Review AI claims against evidence

Open ai-review-prompt.txt. In an organisation-approved AI tool, submit the prompt with the synthetic input and result only. Record the tool/model name, date and prompt version if available. If no approved AI tool is available, manually draft a tentative analyst summary and label it as a human exercise.

For each returned claim, record its cited ID, the actual field value, whether the claim follows, and any correction. Reject conclusions unsupported by the records even when the model is confident. Include one alternative explanation. The AI response is a proposal; it has no authority to approve or execute a security action.

6 Verify the corresponding operational control

On the assigned resource, use IAM > Check access for the test principal and capture the failing operation from the Activity log. Compare the action to the role definition and scope. Propose the smallest grant that resolves the duty. Re-test an allowed operation and an operation outside scope after a trainer-approved change.

For every screenshot or export, record resource scope, UTC time and expected versus observed result. Redact personal data and secrets. If blocked, record the exact error, missing permission/licence, what could be inspected, and the unverified outcome. Ask the trainer to provide an authorised sandbox demonstration where the assessed ability cannot otherwise be evidenced.

7 Submit evidence and clean up

Complete evidence-template.md with baseline, changed result, AI claim review and Azure evidence or clearly labelled limitations. Keep results.json and changed-results.json with your submission. Check that your conclusion distinguishes an observed fact from a hypothesis. Remove only resources or assignments you created with trainer approval; do not delete shared training infrastructure.

Acceptance checks

- The unchanged fixture produces the exact expected report.
- The changed fixture produces the explained result and a different input hash.
- At least one AI or analyst claim is checked against a specific record and field.
- The report states simulation, Azure verified or blocked for each relevant claim.
- The output is a access diagnosis and review decisions with an owner, evidence and remaining uncertainty.

Troubleshooting

- Python not found: install the trainer-approved Python distribution or use py -3 on Windows.
- File not found: open the terminal in this lab folder; check the input filename.
- JSON parse error: validate with python3 -m json.tool changed-data.json; use lowercase true and false in JSON.
- Baseline mismatch: restore the supplied fixture and inspect the first differing decision; never edit the expected file.
- Azure denial or missing feature: capture the exact error and scope. A blocker is a limitation, not proof of competence or deployment.
- AI invents evidence: reject the claim, cite the missing ID, and request an evidence-limited revision.

References

The Learner Guide contains the complete source register and the lecture explanations for this lab. Original examples are adapted from the supplied Azure and AI-security references. Product behaviour should be checked against current Microsoft Learn documentation before a real deployment.

Lab 12 Unauthorized access investigation

AI for IT Security Professionals | TGS-2023039344 | v6.0

Purpose and output

Create a chronological evidence timeline and incident brief. This lab supports LO4. It uses a simplified deterministic teaching model, not a trained AI system or full Azure emulator. The AI review is a separate exercise using the included prompt.

Requirements

Python 3.10 or later, a text editor and this entire folder. No packages, network access or API key are required for the baseline. On Windows use `py -3` if `python3` is unavailable. An approved AI tool is optional; the trainer can provide a review response for critique. Azure exercises require the trainer-assigned subscription, permission and feature entitlement. Record an exact blocker where unavailable; never describe an unperformed test as passed.

Folder contents

- `mock-data.json`: synthetic input with no real personal data or credentials.
- `analyse.py`: runnable analysis for this lab only.
- `expected-results.json`: expected baseline and source checksum.
- `ai-review-prompt.txt`: evidence-constrained AI review prompt.
- `evidence-template.md`: your deliverable template.
- `INSTRUCTIONS.md` and `INSTRUCTIONS.pdf`: the same procedure in two formats.

1 Prepare a working copy

Copy this whole folder to a location you can edit. Open a terminal in the copied folder. Confirm Python and inspect the data before running code.

```
python3 --version
python3 -m json.tool mock-data.json
```

Identify the record IDs and explain the meaning of each field. All values are invented classroom fixtures. Open `analyse.py` in the editor and locate the decision rule. It reads a local JSON file and writes a local report.

2 Run the baseline

```
python3 analyse.py --input mock-data.json --output results.json
```

Open `results.json`. The expected decisions in output order are: `signin`, `role-grant`, `read-data`. The `source_sha256` binds the report to the exact input bytes. Compare against the supplied baseline:

```
python3 -c "import json; a=json.load(open('results.json')); b=json.load(open('expected-results.json')); assert a==b; print('BASELINE PASS')"
```

A pass proves this fixture was processed as specified; it does not prove an Azure control was deployed. Copy the input checksum and decision rows into your evidence template.

3 Trace the mechanism

Evidence led investigation

A hypothesis is tested against events rather than treated as a fact because AI repeats it.

Contract: `hypothesis`, `supporting_ids`, `contradicting_ids`, `status`

Accepted case: An access event and role change support a privilege-use hypothesis.

Counterexample: Only a high anomaly score supports a claimed compromise.

Diagnosis: Collect corroboration and record uncertainty before containment decisions.

Evidence: Maintain a hypothesis table with supported, contradicted and unresolved states.

Incident timeline

Order events by event time while retaining sources and integrity hashes.

Contract: timestamp_utc, event_id, actor, source_hash

Accepted case: E03 role grant precedes E07 resource read.

Counterexample: A summary reverses the order and claims the read caused the grant.

Diagnosis: Rebuild chronology from raw evidence and preserve original files unchanged.

Evidence: The timeline must cite exact event IDs and UTC timestamps.

Containment approval

The analyst chooses a proportionate action and an authorised owner approves its execution.

Contract: incident_id, action, approver, rollback, status

Accepted case: A test account suspension is approved with a recovery owner.

Counterexample: An AI summary directly disables a production administrator.

Diagnosis: Hold execution until approval and scope checks pass; preserve forensic evidence first.

Evidence: Record decision rationale, authority, action result and recovery route.

4 Change one input and test the consequence

Save a copy of mock-data.json as changed-data.json using the editor. Change E07 minute to 2. The read now precedes the role grant. Revise the hypothesis instead of retaining the original causal claim.

```
python3 analyse.py --input changed-data.json --output changed-results.json
```

Compare decisions and source hashes with the baseline. Identify the exact expression in analyse.py responsible for the changed behaviour. Do not overwrite expected-results.json to make a failing baseline pass. Restore the original fixture if you edited it accidentally.

5 Review AI claims against evidence

Open ai-review-prompt.txt. In an organisation-approved AI tool, submit the prompt with the synthetic input and result only. Record the tool/model name, date and prompt version if available. If no approved AI tool is available, manually draft a tentative analyst summary and label it as a human exercise.

For each returned claim, record its cited ID, the actual field value, whether the claim follows, and any correction. Reject conclusions unsupported by the records even when the model is confident. Include one alternative explanation. The AI response is a proposal; it has no authority to approve or execute a security action.

6 Verify the corresponding operational control

Microsoft Sentinel > Incidents > trainer-provided incident. Inspect entities, alerts and source events. Record UTC timestamps and correlation IDs. Compare at least two hypotheses and identify missing evidence. Containment requires a named authorised decision; do not disable real accounts as part of this exercise.

For every screenshot or export, record resource scope, UTC time and expected versus observed result. Redact personal data and secrets. If blocked, record the exact error, missing permission/licence, what could be inspected, and the unverified outcome. Ask the trainer to provide an authorised sandbox demonstration where the assessed ability cannot otherwise be evidenced.

7 Submit evidence and clean up

Complete evidence-template.md with baseline, changed result, AI claim review and Azure evidence or clearly labelled limitations. Keep results.json and changed-results.json with your submission. Check that your conclusion distinguishes an observed fact from a hypothesis. Remove only resources or assignments you created with trainer approval; do not delete shared training infrastructure.

Acceptance checks

- The unchanged fixture produces the exact expected report.
- The changed fixture produces the explained result and a different input hash.
- At least one AI or analyst claim is checked against a specific record and field.
- The report states simulation, Azure verified or blocked for each relevant claim.
- The output is a chronological evidence timeline and incident brief with an owner, evidence and remaining uncertainty.

Troubleshooting

- Python not found: install the trainer-approved Python distribution or use py -3 on Windows.
- File not found: open the terminal in this lab folder; check the input filename.
- JSON parse error: validate with python3 -m json.tool changed-data.json; use lowercase true and false in JSON.
- Baseline mismatch: restore the supplied fixture and inspect the first differing decision; never edit the expected file.
- Azure denial or missing feature: capture the exact error and scope. A blocker is a limitation, not proof of competence or deployment.
- AI invents evidence: reject the claim, cite the missing ID, and request an evidence-limited revision.

References

The Learner Guide contains the complete source register and the lecture explanations for this lab. Original examples are adapted from the supplied Azure and AI-security references. Product behaviour should be checked against current Microsoft Learn documentation before a real deployment.

Lab 13 Prompt injection and retrieval boundaries

AI for IT Security Professionals | TGS-2023039344 | v6.0

Purpose and output

Create a retrieval boundary test and injection analysis. This lab supports LO4. It uses a simplified deterministic teaching model, not a trained AI system or full Azure emulator. The AI review is a separate exercise using the included prompt.

Requirements

Python 3.10 or later, a text editor and this entire folder. No packages, network access or API key are required for the baseline. On Windows use `py -3` if `python3` is unavailable. An approved AI tool is optional; the trainer can provide a review response for critique. Azure exercises require the trainer-assigned subscription, permission and feature entitlement. Record an exact blocker where unavailable; never describe an unperformed test as passed.

Folder contents

- `mock-data.json`: synthetic input with no real personal data or credentials.
- `analyse.py`: runnable analysis for this lab only.
- `expected-results.json`: expected baseline and source checksum.
- `ai-review-prompt.txt`: evidence-constrained AI review prompt.
- `evidence-template.md`: your deliverable template.
- `INSTRUCTIONS.md` and `INSTRUCTIONS.pdf`: the same procedure in two formats.

1 Prepare a working copy

Copy this whole folder to a location you can edit. Open a terminal in the copied folder. Confirm Python and inspect the data before running code.

```
python3 --version
python3 -m json.tool mock-data.json
```

Identify the record IDs and explain the meaning of each field. All values are invented classroom fixtures. Open `analyse.py` in the editor and locate the decision rule. It reads a local JSON file and writes a local report.

2 Run the baseline

```
python3 analyse.py --input mock-data.json --output results.json
```

Open `results.json`. The expected decisions in output order are: RETRIEVE, DENY_TENANT, QUARANTINE_UNTRUSTED. The `source_sha256` binds the report to the exact input bytes. Compare against the supplied baseline:

```
python3 -c "import json; a=json.load(open('results.json')); b=json.load(open('expected-results.json')); assert a==b; print('BASELINE PASS')"
```

A pass proves this fixture was processed as specified; it does not prove an Azure control was deployed. Copy the input checksum and decision rows into your evidence template.

3 Trace the mechanism

Indirect prompt injection

Untrusted retrieved text tries to alter an agent's objective or trigger tools.

Contract: `document_id`, `trust_level`, `requested_tool`

Accepted case: The agent treats a quoted instruction as evidence to summarise.

Counterexample: A retrieved log tells the agent to upload confidential data.

Diagnosis: Enforce tool authorisation outside the model and quarantine suspicious context.

Evidence: Record the injected document ID and denied tool request.

RAG access filtering

Retrieval limits chunks to the requesting user's allowed data before generation.

Contract: user_tenant, chunk_tenant, classification

Accepted case: Tenant A retrieves only tenant A approved chunks.

Counterexample: The model receives tenant B chunks and is told not to repeat them.

Diagnosis: Filter before context assembly; post-generation masking cannot remove prior exposure.

Evidence: Log retrieval filters and test cross-tenant denied queries.

Memory poisoning

Persistent state must have provenance and bounded scope before reuse.

Contract: memory_id, source, tenant_id, expires_at

Accepted case: A verified preference stays within the same user scope.

Counterexample: A malicious document stores an instruction as global policy.

Diagnosis: Reject untrusted writes and invalidate contaminated entries through an audited process.

Evidence: Record writer identity, source trust and memory expiry.

4 Change one input and test the consequence

Save a copy of mock-data.json as changed-data.json using the editor. Replace chunk-c text with an innocuous paraphrase but keep trusted false. It must remain quarantined; this fixture uses provenance rather than a keyword detector.

```
python3 analyse.py --input changed-data.json --output changed-results.json
```

Compare decisions and source hashes with the baseline. Identify the exact expression in analyse.py responsible for the changed behaviour. Do not overwrite expected-results.json to make a failing baseline pass. Restore the original fixture if you edited it accidentally.

5 Review AI claims against evidence

Open ai-review-prompt.txt. In an organisation-approved AI tool, submit the prompt with the synthetic input and result only. Record the tool/model name, date and prompt version if available. If no approved AI tool is available, manually draft a tentative analyst summary and label it as a human exercise.

For each returned claim, record its cited ID, the actual field value, whether the claim follows, and any correction. Reject conclusions unsupported by the records even when the model is confident. Include one alternative explanation. The AI response is a proposal; it has no authority to approve or execute a security action.

6 Verify the corresponding operational control

Draw the approved AI workflow: user identity, retriever, model, tool gateway and audit store. Identify where tenant filtering occurs before the model sees context. If using an approved AI tool, paste only

chunk-a and the synthetic malicious text as quoted data. Record model behaviour separately from gateway enforcement.

For every screenshot or export, record resource scope, UTC time and expected versus observed result. Redact personal data and secrets. If blocked, record the exact error, missing permission/licence, what could be inspected, and the unverified outcome. Ask the trainer to provide an authorised sandbox demonstration where the assessed ability cannot otherwise be evidenced.

7 Submit evidence and clean up

Complete evidence-template.md with baseline, changed result, AI claim review and Azure evidence or clearly labelled limitations. Keep results.json and changed-results.json with your submission. Check that your conclusion distinguishes an observed fact from a hypothesis. Remove only resources or assignments you created with trainer approval; do not delete shared training infrastructure.

Acceptance checks

- The unchanged fixture produces the exact expected report.
- The changed fixture produces the explained result and a different input hash.
- At least one AI or analyst claim is checked against a specific record and field.
- The report states simulation, Azure verified or blocked for each relevant claim.
- The output is a retrieval boundary test and injection analysis with an owner, evidence and remaining uncertainty.

Troubleshooting

- Python not found: install the trainer-approved Python distribution or use py -3 on Windows.
- File not found: open the terminal in this lab folder; check the input filename.
- JSON parse error: validate with python3 -m json.tool changed-data.json; use lowercase true and false in JSON.
- Baseline mismatch: restore the supplied fixture and inspect the first differing decision; never edit the expected file.
- Azure denial or missing feature: capture the exact error and scope. A blocker is a limitation, not proof of competence or deployment.
- AI invents evidence: reject the claim, cite the missing ID, and request an evidence-limited revision.

References

The Learner Guide contains the complete source register and the lecture explanations for this lab. Original examples are adapted from the supplied Azure and AI-security references. Product behaviour should be checked against current Microsoft Learn documentation before a real deployment.

Lab 14 Tool gateway and response approvals

AI for IT Security Professionals | TGS-2023039344 | v6.0

Purpose and output

Create a gateway decision log and approval-binding test. This lab supports LO4. It uses a simplified deterministic teaching model, not a trained AI system or full Azure emulator. The AI review is a separate exercise using the included prompt.

Requirements

Python 3.10 or later, a text editor and this entire folder. No packages, network access or API key are required for the baseline. On Windows use `py -3` if `python3` is unavailable. An approved AI tool is optional; the trainer can provide a review response for critique. Azure exercises require the trainer-assigned subscription, permission and feature entitlement. Record an exact blocker where unavailable; never describe an unperformed test as passed.

Folder contents

- `mock-data.json`: synthetic input with no real personal data or credentials.
- `analyse.py`: runnable analysis for this lab only.
- `expected-results.json`: expected baseline and source checksum.
- `ai-review-prompt.txt`: evidence-constrained AI review prompt.
- `evidence-template.md`: your deliverable template.
- `INSTRUCTIONS.md` and `INSTRUCTIONS.pdf`: the same procedure in two formats.

1 Prepare a working copy

Copy this whole folder to a location you can edit. Open a terminal in the copied folder. Confirm Python and inspect the data before running code.

```
python3 --version
python3 -m json.tool mock-data.json
```

Identify the record IDs and explain the meaning of each field. All values are invented classroom fixtures. Open `analyse.py` in the editor and locate the decision rule. It reads a local JSON file and writes a local report.

2 Run the baseline

```
python3 analyse.py --input mock-data.json --output results.json
```

Open `results.json`. The expected decisions in output order are: ALLOW, DENY_TOOL, DENY_BINDING, DENY_EXPIRED. The `source_sha256` binds the report to the exact input bytes. Compare against the supplied baseline:

```
python3 -c "import json; a=json.load(open('results.json')); b=json.load(open('expected-results.json')); assert a==b; print('BASELINE PASS')"
```

A pass proves this fixture was processed as specified; it does not prove an Azure control was deployed. Copy the input checksum and decision rows into your evidence template.

3 Trace the mechanism

Tool schema validation

A gateway checks required fields and allowed values before calling a privileged API.

Contract: `tool_name`, `target_id`, `action`, `approval_id`

Accepted case: read_incident targets an allowed training incident.

Counterexample: delete_resource is absent from the allowed tool set.

Diagnosis: Deny unknown tools and extra arguments; model instructions do not create permission.

Evidence: Log the validated request and explicit denial reason.

Approval binding

Approval must match the specific action, target and expiry at execution time.

Contract: approval_id, action_hash, target_id, expires_utc

Accepted case: The approved hash matches the queued action.

Counterexample: A different target is substituted after approval.

Diagnosis: Invalidate mismatched approvals and require a fresh decision for the changed action.

Evidence: Compare approved and executed action digests.

Idempotent response

An idempotency key prevents repeated delivery from duplicating a response action.

Contract: incident_id, action_type, idempotency_key

Accepted case: Two identical requests produce one ticket.

Counterexample: A retry creates duplicate disable actions with conflicting states.

Diagnosis: Store action outcome and reconcile before retrying a timed-out request.

Evidence: Record request key, first result and duplicate suppression count.

4 Change one input and test the consequence

Save a copy of mock-data.json as changed-data.json using the editor. Change call-c approved_target to INC-002. It becomes ALLOW in the fixture; explain why a real approval must come from an authenticated approver and cannot be edited by the requester.

```
python3 analyse.py --input changed-data.json --output changed-results.json
```

Compare decisions and source hashes with the baseline. Identify the exact expression in analyse.py responsible for the changed behaviour. Do not overwrite expected-results.json to make a failing baseline pass. Restore the original fixture if you edited it accidentally.

5 Review AI claims against evidence

Open ai-review-prompt.txt. In an organisation-approved AI tool, submit the prompt with the synthetic input and result only. Record the tool/model name, date and prompt version if available. If no approved AI tool is available, manually draft a tentative analyst summary and label it as a human exercise.

For each returned claim, record its cited ID, the actual field value, whether the claim follows, and any correction. Reject conclusions unsupported by the records even when the model is confident. Include one alternative explanation. The AI response is a proposal; it has no authority to approve or execute a security action.

6 Verify the corresponding operational control

Review a proposed Sentinel automation rule and Logic Apps playbook in the training environment. Identify trigger, managed identity, allowed action, target and human approval step. Keep it disabled during review. The local script validates fixtures only and never calls a cloud API.

For every screenshot or export, record resource scope, UTC time and expected versus observed result. Redact personal data and secrets. If blocked, record the exact error, missing permission/licence, what could be inspected, and the unverified outcome. Ask the trainer to provide an authorised sandbox demonstration where the assessed ability cannot otherwise be evidenced.

7 Submit evidence and clean up

Complete evidence-template.md with baseline, changed result, AI claim review and Azure evidence or clearly labelled limitations. Keep results.json and changed-results.json with your submission. Check that your conclusion distinguishes an observed fact from a hypothesis. Remove only resources or assignments you created with trainer approval; do not delete shared training infrastructure.

Acceptance checks

- The unchanged fixture produces the exact expected report.
- The changed fixture produces the explained result and a different input hash.
- At least one AI or analyst claim is checked against a specific record and field.
- The report states simulation, Azure verified or blocked for each relevant claim.
- The output is a gateway decision log and approval-binding test with an owner, evidence and remaining uncertainty.

Troubleshooting

- Python not found: install the trainer-approved Python distribution or use py -3 on Windows.
- File not found: open the terminal in this lab folder; check the input filename.
- JSON parse error: validate with python3 -m json.tool changed-data.json; use lowercase true and false in JSON.
- Baseline mismatch: restore the supplied fixture and inspect the first differing decision; never edit the expected file.
- Azure denial or missing feature: capture the exact error and scope. A blocker is a limitation, not proof of competence or deployment.
- AI invents evidence: reject the claim, cite the missing ID, and request an evidence-limited revision.

References

The Learner Guide contains the complete source register and the lecture explanations for this lab. Original examples are adapted from the supplied Azure and AI-security references. Product behaviour should be checked against current Microsoft Learn documentation before a real deployment.

Lab 15 AI security evaluation

AI for IT Security Professionals | TGS-2023039344 | v6.0

Purpose and output

Create a confusion matrix and holdout evaluation plan. This lab supports LO1. It uses a simplified deterministic teaching model, not a trained AI system or full Azure emulator. The AI review is a separate exercise using the included prompt.

Requirements

Python 3.10 or later, a text editor and this entire folder. No packages, network access or API key are required for the baseline. On Windows use `py -3` if `python3` is unavailable. An approved AI tool is optional; the trainer can provide a review response for critique. Azure exercises require the trainer-assigned subscription, permission and feature entitlement. Record an exact blocker where unavailable; never describe an unperformed test as passed.

Folder contents

- `mock-data.json`: synthetic input with no real personal data or credentials.
- `analyse.py`: runnable analysis for this lab only.
- `expected-results.json`: expected baseline and source checksum.
- `ai-review-prompt.txt`: evidence-constrained AI review prompt.
- `evidence-template.md`: your deliverable template.
- `INSTRUCTIONS.md` and `INSTRUCTIONS.pdf`: the same procedure in two formats.

1 Prepare a working copy

Copy this whole folder to a location you can edit. Open a terminal in the copied folder. Confirm Python and inspect the data before running code.

```
python3 --version
python3 -m json.tool mock-data.json
```

Identify the record IDs and explain the meaning of each field. All values are invented classroom fixtures. Open `analyse.py` in the editor and locate the decision rule. It reads a local JSON file and writes a local report.

2 Run the baseline

```
python3 analyse.py --input mock-data.json --output results.json
```

Open `results.json`. The expected decisions in output order are: `REVIEW_ERRORS`. The `source_sha256` binds the report to the exact input bytes. Compare against the supplied baseline:

```
python3 -c "import json; a=json.load(open('results.json')); b=json.load(open('expected-results.json')); assert a==b; print('BASELINE PASS')"
```

A pass proves this fixture was processed as specified; it does not prove an Azure control was deployed. Copy the input checksum and decision rows into your evidence template.

3 Trace the mechanism

AI evaluation holdout

Use separate tuning and holdout examples to estimate performance on unseen cases.

Contract: `dataset_split`, `label`, `prediction`, `threshold`

Accepted case: The threshold is chosen on tuning data and frozen for holdout.

Counterexample: The holdout labels are used to optimise the same threshold.

Diagnosis: Create a fresh holdout and document the data boundary; estimates from contaminated data are optimistic.

Evidence: Retain split membership, fixed threshold and error analysis.

Adversarial input resilience

Evaluate both benign and malicious context against the same external control boundary.

Contract: case_id, attack_type, expected_action, observed_action

Accepted case: A malicious instruction still produces a denied privileged action.

Counterexample: A harmless-looking paraphrase bypasses a keyword-only filter.

Diagnosis: Broaden the test set and enforce authorisation independent of keyword detection.

Evidence: Report bypasses and false alarms with the exact test-case IDs.

Model and dependency provenance

An approved release binds model version, prompt template and dependency checksums.

Contract: model_version, prompt_hash, package_hash, approval

Accepted case: All artefacts match the approved release manifest.

Counterexample: An unreviewed package changes tool behaviour.

Diagnosis: Block rollout and restore the approved artifact set; signature and origin checks complement scanning.

Evidence: Keep the manifest and a verified reproducibility record.

4 Change one input and test the consequence

Save a copy of mock-data.json as changed-data.json using the editor. Change case-3 predicted to true. Recall becomes 1.0 while precision becomes 0.75. Explain why this single fixture edit is not proof of an improved detector on unseen data.

```
python3 analyse.py --input changed-data.json --output changed-results.json
```

Compare decisions and source hashes with the baseline. Identify the exact expression in analyse.py responsible for the changed behaviour. Do not overwrite expected-results.json to make a failing baseline pass. Restore the original fixture if you edited it accidentally.

5 Review AI claims against evidence

Open ai-review-prompt.txt. In an organisation-approved AI tool, submit the prompt with the synthetic input and result only. Record the tool/model name, date and prompt version if available. If no approved AI tool is available, manually draft a tentative analyst summary and label it as a human exercise.

For each returned claim, record its cited ID, the actual field value, whether the claim follows, and any correction. Reject conclusions unsupported by the records even when the model is confident. Include one alternative explanation. The AI response is a proposal; it has no authority to approve or execute a security action.

6 Verify the corresponding operational control

Use the labelled synthetic cases only. Record model/prompt version if an approved model is evaluated. Keep the labels out of the prediction prompt, freeze the prompt before the holdout run, and compare case IDs. Do not label the deterministic fixture runner as a trained AI detector.

For every screenshot or export, record resource scope, UTC time and expected versus observed result. Redact personal data and secrets. If blocked, record the exact error, missing permission/licence, what could be inspected, and the unverified outcome. Ask the trainer to provide an authorised sandbox demonstration where the assessed ability cannot otherwise be evidenced.

7 Submit evidence and clean up

Complete evidence-template.md with baseline, changed result, AI claim review and Azure evidence or clearly labelled limitations. Keep results.json and changed-results.json with your submission. Check that your conclusion distinguishes an observed fact from a hypothesis. Remove only resources or assignments you created with trainer approval; do not delete shared training infrastructure.

Acceptance checks

- The unchanged fixture produces the exact expected report.
- The changed fixture produces the explained result and a different input hash.
- At least one AI or analyst claim is checked against a specific record and field.
- The report states simulation, Azure verified or blocked for each relevant claim.
- The output is a confusion matrix and holdout evaluation plan with an owner, evidence and remaining uncertainty.

Troubleshooting

- Python not found: install the trainer-approved Python distribution or use py -3 on Windows.
- File not found: open the terminal in this lab folder; check the input filename.
- JSON parse error: validate with python3 -m json.tool changed-data.json; use lowercase true and false in JSON.
- Baseline mismatch: restore the supplied fixture and inspect the first differing decision; never edit the expected file.
- Azure denial or missing feature: capture the exact error and scope. A blocker is a limitation, not proof of competence or deployment.
- AI invents evidence: reject the claim, cite the missing ID, and request an evidence-limited revision.

References

The Learner Guide contains the complete source register and the lecture explanations for this lab. Original examples are adapted from the supplied Azure and AI-security references. Product behaviour should be checked against current Microsoft Learn documentation before a real deployment.

Lab 16 Incident response and service recovery

AI for IT Security Professionals | TGS-2023039344 | v6.0

Purpose and output

Create a response state register and closure evidence. This lab supports LO4. It uses a simplified deterministic teaching model, not a trained AI system or full Azure emulator. The AI review is a separate exercise using the included prompt.

Requirements

Python 3.10 or later, a text editor and this entire folder. No packages, network access or API key are required for the baseline. On Windows use `py -3` if `python3` is unavailable. An approved AI tool is optional; the trainer can provide a review response for critique. Azure exercises require the trainer-assigned subscription, permission and feature entitlement. Record an exact blocker where unavailable; never describe an unperformed test as passed.

Folder contents

- `mock-data.json`: synthetic input with no real personal data or credentials.
- `analyse.py`: runnable analysis for this lab only.
- `expected-results.json`: expected baseline and source checksum.
- `ai-review-prompt.txt`: evidence-constrained AI review prompt.
- `evidence-template.md`: your deliverable template.
- `INSTRUCTIONS.md` and `INSTRUCTIONS.pdf`: the same procedure in two formats.

1 Prepare a working copy

Copy this whole folder to a location you can edit. Open a terminal in the copied folder. Confirm Python and inspect the data before running code.

```
python3 --version
python3 -m json.tool mock-data.json
```

Identify the record IDs and explain the meaning of each field. All values are invented classroom fixtures. Open `analyse.py` in the editor and locate the decision rule. It reads a local JSON file and writes a local report.

2 Run the baseline

```
python3 analyse.py --input mock-data.json --output results.json
```

Open `results.json`. The expected decisions in output order are: `RECOVERED`, `WAIT_APPROVAL`, `VERIFY_REQUIRED`. The `source_sha256` binds the report to the exact input bytes. Compare against the supplied baseline:

```
python3 -c "import json; a=json.load(open('results.json')); b=json.load(open('expected-results.json')); assert a==b; print('BASELINE PASS')"
```

A pass proves this fixture was processed as specified; it does not prove an Azure control was deployed. Copy the input checksum and decision rows into your evidence template.

3 Trace the mechanism

Response playbook state

A playbook advances through validated states with a human gate before containment.

Contract: new, triaged, approved, contained, recovered

Accepted case: An approved training incident reaches contained after a successful action.

Counterexample: A failed action is recorded as contained.

Diagnosis: Keep failed actions pending investigation and reconcile actual service state.

Evidence: Record each transition, actor, timestamp and result.

Security service health

Missing telemetry can make apparent alert reduction a detection failure.

Contract: expected_events, observed_events, connector_status

Accepted case: 95 of 100 expected heartbeat events meet a 90% classroom threshold.

Counterexample: Zero alerts coincide with a disconnected source.

Diagnosis: Restore ingestion, backfill where possible and state the monitoring gap.

Evidence: Measure completeness and delay alongside alert counts.

Closure and residual risk

Close an incident only when remediation and verification evidence address its hypothesis.

Contract: root_cause, control_change, retest, residual_risk

Accepted case: A role correction passes allowed and denied access tests.

Counterexample: A ticket closes because the AI report sounds confident.

Diagnosis: Reopen unsupported closure and assign remaining risk to an accountable owner.

Evidence: Archive the evidence bundle, change reference and signed decision.

4 Change one input and test the consequence

Save a copy of mock-data.json as changed-data.json using the editor. Set INC-003 retest_ok to true. It must still remain VERIFY_REQUIRED because telemetry coverage is zero.

```
python3 analyse.py --input changed-data.json --output changed-results.json
```

Compare decisions and source hashes with the baseline. Identify the exact expression in analyse.py responsible for the changed behaviour. Do not overwrite expected-results.json to make a failing baseline pass. Restore the original fixture if you edited it accidentally.

5 Review AI claims against evidence

Open ai-review-prompt.txt. In an organisation-approved AI tool, submit the prompt with the synthetic input and result only. Record the tool/model name, date and prompt version if available. If no approved AI tool is available, manually draft a tentative analyst summary and label it as a human exercise.

For each returned claim, record its cited ID, the actual field value, whether the claim follows, and any correction. Reject conclusions unsupported by the records even when the model is confident. Include one alternative explanation. The AI response is a proposal; it has no authority to approve or execute a security action.

6 Verify the corresponding operational control

Review the training incident and related automation run history. Record each action result and the connector health. Compare pre/post control evidence and a negative access test. Close only the assigned simulated incident when the evidence is complete; record residual risk and restore any lab-created configuration under trainer supervision.

For every screenshot or export, record resource scope, UTC time and expected versus observed result. Redact personal data and secrets. If blocked, record the exact error, missing permission/licence, what could be inspected, and the unverified outcome. Ask the trainer to provide an authorised sandbox demonstration where the assessed ability cannot otherwise be evidenced.

7 Submit evidence and clean up

Complete evidence-template.md with baseline, changed result, AI claim review and Azure evidence or clearly labelled limitations. Keep results.json and changed-results.json with your submission. Check that your conclusion distinguishes an observed fact from a hypothesis. Remove only resources or assignments you created with trainer approval; do not delete shared training infrastructure.

Acceptance checks

- The unchanged fixture produces the exact expected report.
- The changed fixture produces the explained result and a different input hash.
- At least one AI or analyst claim is checked against a specific record and field.
- The report states simulation, Azure verified or blocked for each relevant claim.
- The output is a response state register and closure evidence with an owner, evidence and remaining uncertainty.

Troubleshooting

- Python not found: install the trainer-approved Python distribution or use py -3 on Windows.
- File not found: open the terminal in this lab folder; check the input filename.
- JSON parse error: validate with python3 -m json.tool changed-data.json; use lowercase true and false in JSON.
- Baseline mismatch: restore the supplied fixture and inspect the first differing decision; never edit the expected file.
- Azure denial or missing feature: capture the exact error and scope. A blocker is a limitation, not proof of competence or deployment.
- AI invents evidence: reject the claim, cite the missing ID, and request an evidence-limited revision.

References

The Learner Guide contains the complete source register and the lecture explanations for this lab. Original examples are adapted from the supplied Azure and AI-security references. Product behaviour should be checked against current Microsoft Learn documentation before a real deployment.

Source register

The rebuilt material uses original explanations and synthetic cases derived from the supplied reference library. The legacy Azure deck carries TGS-2023039344 and establishes the identity, network, compute, data and SecOps coverage floor. Its two-day schedule is superseded by the current four-day course and the approved proposal's 30 training hours plus 2 assessment hours.

- Microsoft Azure Security Engineer Associate (AZ-500)-v5.pptx, supplied reference, 53 slides: original Azure controls and lab coverage.

- AI-Security-for-AI-Agents.pptx, supplied reference, 10 slides: agent trust boundaries, tool misuse, memory and orchestration controls.
- Borges and Campbell, AI Security Engineering: Securing Agentic Systems in Production, supplied early-release excerpt, 48 PDF pages: agentic systems and operational security boundaries.
- Gupta and Mittal, Foundations of Modern Information Security, supplied as AI Security 2.pdf, 349 PDF pages: cloud, data, identity, governance and operational security.
- Ashish Rajan, AI Security Engineering: Design, Build, and Secure Dependable AI Systems, supplied as AI Security Engineering3.pdf, 370 PDF pages: AI threat landscape, pipelines, production controls, monitoring and governance.
- Ex_Files_Cybersecurity_Policy_Governance, supplied exercise library: policy evidence and accountable control ownership. Original exercise files remain in the reference library.
- Current LMS-linked WA and PP papers, recovered from the supplied Drive folder and matched by file ID to the authenticated LMS record: 9 questions K1–K9; 3 tasks A1–A6; 60 minutes each. The old instrument headings use the previous Azure title and the same course code.

Current primary-source checks, 6 September 2026:

- Azure RBAC: <https://learn.microsoft.com/en-us/azure/role-based-access-control/overview>
- NSG rule evaluation: <https://learn.microsoft.com/en-us/azure/virtual-network/network-security-groups-overview>
- Conditional Access report-only: <https://learn.microsoft.com/en-us/entra/identity/conditional-access/concept-conditional-access-report-only>
- AI agent control boundaries:
https://cheatsheetseries.owasp.org/cheatsheets/AI_Agent_Security_Cheat_Sheet.html

These product references support the corresponding mechanisms. Classroom algorithms and thresholds are explicitly illustrative; the scripts do not implement full Azure policy, identity or network semantics. Review current documentation and the assigned tenant before a real deployment.